Distributed Link Sparsification for Scalable Scheduling Using Graph Neural Networks

Zhongyuan Zhao, *Member, IEEE*, Gunjan Verma, *Member, IEEE*, Ananthram Swami, *Life Fellow, IEEE*, and Santiago Segarra, *Senior Member, IEEE*

***Abstract*—In wireless networks characterized by dense connectivity, the significant signaling overhead generated by distributed link scheduling algorithms can exacerbate issues like congestion, energy consumption, and radio footprint expansion. To mitigate these challenges, we propose a distributed link sparsification scheme employing graph neural networks (GNNs) to reduce scheduling overhead for delay-tolerant traffic while maintaining network capacity. A GNN module is trained to adjust contention thresholds for individual links based on traffic statistics and network topology, enabling links to withdraw from scheduling contention when they are unlikely to succeed. Our approach is facilitated by a novel offline constrained unsupervised learning algorithm capable of balancing two competing objectives: minimizing scheduling overhead while ensuring that total utility meets the required level. In simulated wireless multi-hop networks with up to 500 links, our link sparsification technique effectively alleviates network congestion and reduces radio footprints across four distinct distributed link scheduling protocols.**

***Index Terms*—Threshold, massive access, scalable scheduling, graph neural networks, constrained unsupervised learning.**

I. Introduction

THE proliferation of wireless devices and emerging machine-type communications (MTC) [2] has led to new requirements for next-generation wireless networks, including massive access in ultra-dense networks, spectrum and energy efficiencies, multi-hop connectivity, and scalability [3], [4], [5], [6]. A promising solution to these challenges is self-organizing wireless multi-hop networks, which have been applied to scenarios where infrastructure is infeasible or overloaded, such as military communications, satellite communications, vehicular/drone networks, Internet of Things (IoT), and 5G/6G (device-to-device (D2D), wireless backhaul, integrated access and backhaul (IAB)) [3], [4], [5], [6], [7], [8], [9], [10]. In ultra-dense cellular IoT networks [6], delay-tolerant MTC could be enabled by using cluster heads that first aggregate data from their nearby IoT devices based on distributed clustering and link scheduling [11], [12], then relay data to the base-station either directly or over multiple hops. Mobile MTC, such as vehicular networks and Internet of Battlefield Things [3], can also benefit from the congestion management of self-organizing wireless multi-hop networks. The key enabler of self-organizing wireless multi-hop networks is distributed resource allocation. This includes link scheduling, which determines which links should transmit and when they should transmit [13], [14]. In this work, we focus on link scheduling in wireless multi-hop networks with orthogonal multiple access (OMA). Compared to non-orthogonal multiple access (NOMA) [5], OMA requires less sophisticated transceivers and coordination, therefore, is more friendly to MTC of low-end devices in shared spectrum [6] or harsh/hostile environments [3]. In 5G/6G, OMA is also better suited for D2D, wireless backhaul, and IAB [4], [10].

In wireless multi-hop networks with OMA, link scheduling is typically formulated as solving a maximum weighted independent set (MWIS) problem on a conflict graph [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22]. In a conflict graph, a vertex represents a link in the wireless network, an edge captures the interference relationship between two links, and the vertex weight is the utility of scheduling the corresponding link. Known to be NP-hard [16], [21], the MWIS problem is solved approximately in practical schedulers. Common distributed heuristics include distributed greedy MaxWeight schedulers [17], [18], message passing algorithms [19], [20], their hybrids [12], [22], and randomized algorithms like carrier sense multiple access (CSMA) [23], [24], [25], many of which are inspired by Luby's algorithm [26].

The overhead of distributed link scheduling, referred to as the signaling overhead in MaxWeight schedulers [12], [17], [18], [22] and collision rates and idle listening in CSMA [23], [24], [25], increases with the number of contending neighbors in the network. In dense networks characterized by large contention neighborhoods, the scheduling overhead can be substantial, reducing network capacity [2], [5] while increasing congestion, instability, energy consumption, radio footprint, and security vulnerability in network operations [5], [27], [28], [29]. While existing approaches like topology control by adjusting antenna directivity and transmit power [29], [30], [31], sleep scheduling [28], [31], [32], [33], cross-layer optimization [34], [35], [36], and network coding [37] can partially mitigate challenges of dense wireless networks,

Received 27 February 2024; revised 20 January 2025, 17 June 2025, and 13 August 2025; accepted 1 September 2025. Date of publication 12 September 2025; date of current version 22 December 2025. This work was supported by the U.S. Army Combat Capabilities Development Command (DEVCOM) Army Research Laboratory (ARL) Army Research Office and Accomplished under Cooperative Agreement W911NF-19-2-0269. An earlier version of this paper was presented at [DOI: 10.1109/ICASSP43922.2022.9747437]. The associate editor coordinating the review of this article and approving it for publication was P. Hu. *(Corresponding author: Zhongyuan Zhao.)*

Zhongyuan Zhao and Santiago Segarra are with the Department of Electrical and Computer Engineering, Rice University, Houston, TX 77005 USA (e-mail: zhongyuan.zhao@rice.edu; segarra@rice.edu).

Gunjan Verma and Ananthram Swami are with the U.S. Army DEVCOM Army Research Laboratory (ARL), Adelphi, MD 20783 USA (e-mail: gunjan.verma.civ@army.mil; ananthram.swami.civ@army.mil).

Digital Object Identifier 10.1109/TWC.2025.3606741

1536-1276 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

they often fall short when dealing with the specific issue of massive access.

In this work, we seek scalable scheduling schemes for delay-tolerant traffic to reduce the scheduling overhead in dense wireless networks. Specifically, we introduce an additional step of *link sparsification* into distributed link scheduling to limit the number of participants in the transmission contention by filtering out links unlikely to win, such as those with utilities below certain thresholds [1]. A *baseline* approach is to set a global threshold $u^{(\eta)}$ across the network based on the empirical cumulative distribution function (eCDF) of the per-link utility and a prescribed cut-off quantile η. On an arbitrary link v, such a baseline sparsification can be expressed as

$$h\big(u(v,t)\big) = u(v,t)H\left(u(v,t) - u^{(\eta)}\right) , \tag{1}$$

where $u(v,t) \geq 0$ is the utility of scheduling link v at time t and $H(\cdot)$ is the Heaviside (step) function. If $u(v,t) \leq u^{(\eta)}$, $h\big(u(v,t)\big) = 0$, and link v is muted for time step t without contention so as to reduce scheduling overhead. Since link utility is usually a function of queue length [17], [18] or sojourn time [38], every link will eventually contest for scheduling as the backlog or waiting time grows. The increasing bandwidth of air-interfaces [2] and the capacity of on-chip memory make it feasible to trade off some latency for more connectivity by transmitting larger data payloads less frequently.

Can we improve upon this (statistical) baseline? Ideally, we could refine the global threshold in (1) for each link based on its local traffic and topological information. For instance, lowering the thresholds for links with fewer contending neighbors, and vice versa, can increase transmission opportunities in areas with lower scheduling overhead while reducing contentions in denser areas, potentially leading to improved resource efficiency in the network. Technically, such refinement can be implemented using a vector $\mathbf{z} \in \mathbb{R}_+^{|\mathcal{V}|}$, which scales the global threshold individually for each link in the network $v \in \mathcal{V}$,

$$h_v\big(u(v,t); z\big) = u(v,t)\, H\left(u(v,t) - z(v)u^{(\eta)}\right) , \tag{2}$$

where $z(v) = \mathbf{z}_v$ is the vth element of vector $\mathbf{z}$. Vector $\mathbf{z}$ can be optimized to produce a policy that is Pareto superior to the statistical baseline, achieving at least the same total utility while reducing scheduling overhead across the network.

To achieve this goal, we propose to train a GNN to generate topology-aware multipliers $\mathbf{z}$ for the localized link sparsification $\{h_v(\cdot)\}_{v\in\mathcal{V}}$ in (2). Compared to conventional optimization methods or deep neural networks (DNNs), GNNs have several unique advantages: 1) GNNs are designed to process relational data, including the conflict relationships between wireless links, which serve as key inputs for threshold refinement. 2) GNNs can operate in a fully distributed manner via message passing between neighboring nodes, eliminating the need to aggregate the full network state at a centralized server, which is critical to scalability and robustness. 3) The built-in permutation equivariance of GNNs aligns naturally with tasks in wireless networks–the output of GNNs remains consistent under permutations of input node ordering– making them more efficient in data and structure compared to DNNs, which must learn such symmetry from data. 4) GNNs also generalize well to various network topologies [39], allowing a trained GNN to seamlessly scale to larger networks and adapt to node mobility in wireless networks.

However, applying GNNs to link sparsification faces unique challenges. First, GNNs' reliance on message passing between neighboring devices inherently conflicts with our very purpose of minimizing signaling overhead, making it counterproductive to frequently call GNNs [40]. Our solution enables GNNs to operate infrequently and asynchronously. Another challenge is to design a training scheme that can train GNNs in non-differentiable pipelines, similar to [11], [12], and [22], while optimizing two competing objectives, the total utility and scheduling overhead. To overcome these challenges, we propose a novel, first-order constrained unsupervised learning algorithm that can outperform the standard approach of zeroth-order optimization (ZOO) [41]. Although the training is offline and centralized in simulated environments, the trained GNNs and the sparse schedulers can be deployed to infrastructureless networks and executed in a fully distributed manner.

Contribution. The major contributions of this paper are:

- We propose the first GNN-based distributed link sparsification for scalable link scheduling in dense networks by exploiting the topology of the interference graph.
- We develop alternating stochastic gradient descent (Alt-SGD), a first-order constrained unsupervised learning algorithm that can train the GNNs in a non-differentiable pipeline while balancing two competing objectives.
- Through simulation, we demonstrate that our GNN-based threshold policy can outperform the statistical baseline in dense wireless networks with four different scheduling protocols, and can generalize across topologies.
- We provide practical guidelines on implementing contention and conflict graph construction simultaneously, asynchronous operations for GNNs, gathering the eCDF of per-link utility, and setting the global cut-off quantile.

Notation: We adopt the following notational convention: $(\cdot)^\top$, $\odot$, and $|\cdot|$ respectively represent transpose operator, Hadamard (element-wise) product operator, and the cardinality of a set. $\mathbb{1}(\cdot)$ is the indicator function, and $\mathbb{E}(\cdot)$ stands for expectation. Upright bold lower-case symbol, e.g., $\mathbf{x}$, denotes a column vector, and $\mathbf{x}_i$ denotes the ith element of vector $\mathbf{x}$. Upright bold upper-case symbol $\mathbf{X}$ denotes a matrix, of which the element at row i and column j is denoted by $\mathbf{X}_{ij}$, the entire row i by $\mathbf{X}_{i*}$, and the entire column j by $\mathbf{X}_{*j}$. The major notations, though not exhaustive, are described in Table II.

II. Related Work

A. Scalable Resource Allocation

Scalable link scheduling has been a long-standing challenge for wireless multi-hop networks, where centralized scheduling is hindered by the high communication overhead of gathering the full network state to a server. Distributed scheduling [12], [13], [15], [16], [17], [18], [19], [20], [21], [22], [23], [25] addresses the scalability concerns by tackling the MWIS problem (or its dual, maximum weighted matching) through

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

parallel local contentions among interfering neighbors. These contentions can take either deterministic forms, relying on exchange and comparison of utilities among neighboring links [12], [17], [18], [22], or randomized forms, employing weighted CSMA [23], [25], [42], [43]. Classical distributed schedulers [17], [18] are capable of solving the MWIS problem in $\mathcal{O}(\log|\mathcal{V}|)$ rounds of local message exchange, where $|\mathcal{V}|$ is the number of links in the network. Scheduling within $\mathcal{O}(1)$ (constant) rounds of local exchange can be achieved by truncating the iterations in [17] and [18] or by employing one round of local exchange followed by weighted CSMA [23], [25], [42], [43]. However, for ultra-dense networks, the reduction in rounds of local exchanges is insufficient, since the overhead of each round of local exchange (or the collision rate in CSMA) increases with the conflict degree (the number of interfering neighbors), potentially reaching prohibitively high levels. In contrast to these approaches, our link sparsification scheme lowers the scheduling overhead by reducing the conflict degree. While certain strategies in our approach, such as *buffer-and-transmit* [44] and probabilistic participation in contention [25] are not novel, our method distinguishes itself by eliminating the need for message exchange across the entire neighborhood at every time step, a requirement present in prior models [25], [44].

Besides scheduling, other resource allocation schemes can also help reduce the signaling overhead in wireless multi-hop networks. Topology control [29], [30], [31] seeks to maintain a connected topology of the network with minimum energy consumption by adjusting antenna beams and/or transmit power. In dense networks, topology control can reduce the scheduling overhead by limiting the neighborhood size, at the cost of increased overhearing, latency, and risks of congestion at some critical nodes. Sleep scheduling [28], [31], [32], [33] puts devices with very low traffic demand into periodic sleep mode in a standalone or coordinated manner. It mainly aims to extend the battery lifetime of wireless devices, but can also reduce the densities of connectivity and interference at a given moment. In comparison, link sparsification can benefit active devices with higher traffic demands. Cross-layer optimization [34], [35], [36] jointly optimizes power control, link scheduling, and routing to improve energy efficiency and latency, but it generally entails similar or higher signaling overhead than the aforementioned MWIS schedulers, and may also require centralized computing [35], [36]. Network coding [37] seeks efficient utilization of network capacity by aggregating data packets from different sources destined to the same sink, rather than forwarding them separately, which can also reduce the level of contention. However, the aforementioned methods by themselves are generally insufficient for scalable resource allocation in ultra-dense networks, whereas link sparsification can *complement* them. Lastly, queueing systems with threshold policy have been studied for decades [45], [46] and applied to energy saving for wireless sensor/IoT networks [47], [48]. However, these steady state analyses of queueing systems are not applicable to link scheduling with queues subject to interference constraints, where the service rates (link rates) are influenced by the thresholds through scheduling overhead rather than following a stationary distribution.

B. Machine Learning in Wireless Networks

Machine learning (ML) for wireless networks faces four major challenges. Firstly, while data is abundant in real or simulated systems, labeling it is often difficult [6]. Consequently, learning without labeling is often preferred over supervised and semi-supervised learning in wireless systems. Secondly, due to the limited computing capability, memory, or power supply of resource-constrained wireless devices, lightweight ML models are favored [6]. Thirdly, the distributed and dynamic nature of wireless networks restricts the available ML models and optimization techniques to a few options, such as GNNs [49], multi-agent reinforcement learning (MARL) [50], and distributed optimization [51]. Although typically trained in a centralized and offline manner, GNNs can easily scale to large dynamic networks and generalize to different network topologies. This is because the same trained model of neural networks, which are often relatively lightweight, is shared by all nodes (or those of the same class).

Lastly, ML for wireless resource allocation also faces the challenge of dealing with various constraints. Soft constraints, which could be violated at a penalty, allow policies of distributed resource allocation to be directly generated by GNNs trained through primal-dual optimization [40] or imitation learning [52]. However, for combinatorial optimization (CO) formulated as mixed integer programming, ML systems must adopt certain algorithmic frameworks to ensure the hard constraints are always satisfied. Unfortunately, well-established sequential frameworks like Q-learning [53] and Monte-Carlo tree search [54] are not applicable to distributed systems. A hybrid ML pipeline [11], [12], [22] can fill this gap by using a GNN to encode the network state into the inputs of a classical distributed algorithm, which outputs final decisions and guarantees the observance of constraints on the decision variables. Since many distributed algorithms are non-differentiable, training of GNNs in such hybrid ML pipelines used to rely on ZOO [41]. Recently, first-order deterministic policy gradient (DPG) [55] and graph-based DPG (GDPG) [11], [12] have been applied to train GNNs for link scheduling and packet routing. While our sparse scheduler is similar to GCN-LGS [12] in the use of non-differentiable operations and proxy function for gradient estimation, our learning algorithm takes one step further by simultaneously targeting two competing objectives: to minimize the signaling overhead while guaranteeing the total utility meets a minimum requirement.

III. System Model and Problem Statement

We model a wireless multi-hop network as a connectivity graph $\mathcal{G}^n = (\mathcal{X}, \mathcal{V})$, where each node $i \in \mathcal{X}$ is a transceiver, and an edge $(i, j) \in \mathcal{V}$ represents a directed link from transmitter i to receiver j over-the-air (OTA). The sets of outgoing and incoming links of device i are $\kappa_i = \{(i, j)|(i, j) \in \mathcal{V}\}$ and $\rho_i = \{(j, i)|(j, i) \in \mathcal{V}\}$. Alternatively, edge $(i, j) \in \mathcal{V}$ can also represent a bidirectional link between transceivers i and j. Our approach is applicable to both cases. For each link, there is a queuing system q for packet backlogs.

To describe the scheduling algorithm, we define the *conflict graph*, $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, as follows: a vertex $v \in \mathcal{V}$ represents a

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

link in the wireless network, and the presence of an undirected edge $e = (v_a, v_b) \in \mathcal{E}$ captures the interference relationship between links $v_a, v_b \in \mathcal{V}$, which is considered to follow a physical distance model [16]. For example, two links interfere with each other if their incident nodes are within a certain distance such that their simultaneous transmission would cause the outage probability to exceed a prescribed level, or they share the same node equipped with single radio interface. For the rest of this paper, we assume the conflict graph $\mathcal{G}$ is known; see Section IV-B for its estimation. We consider a simplified scenario in which all the nodes transmit at power levels that are time-invariant. In principle, the interference zone of each link (hence the entire $\mathcal{G}$) depends on the transmit power and directivity of the corresponding nodes. Based on the definition of conflict graph $\mathcal{G}$, a legal schedule must be a set of wireless links that can be activated simultaneously subject to the constraint of spatial OMA, which forms an independent (vertex) set $\boldsymbol{v}$ on $\mathcal{G}$, defined as a set of nodes with no edges connecting each other. See [12, Sec. III-C] for multi-channel OMA extensions in time, frequency, and/or code dimensions.

The *network state* at time t, denoted as $(\mathcal{G}(t), \mathbf{u}(t))$, comprises the conflict graph $\mathcal{G}(t)$ that could change due to node mobility and a utility vector $\mathbf{u}(t)$ collecting $u(v,t) \in \mathbb{R}_+$ for all nodes $v \in \mathcal{V}$. The utility $u(v,t)$ could capture the backlogs $q(v,t)$, link rate $r(v,t)$, sojourn time, and priority of the wireless link v. We denote by $c(\cdot)$ the scheduling contention process that maps every network state into an independent set on the conflict graph, such as MaxWeight [17], [18] and CSMA [23], [24], [25] schedulers. For notational simplicity, we henceforth omit t for operations in the same time slot, and denote the total utility of a vertex set $\boldsymbol{v}$ by $u(\boldsymbol{v}) = \sum_{v \in \boldsymbol{v}} u(v)$.

Notice that in practice, global knowledge of network state is never required by any devices, since the distributed executions of GNNs, link sparsification, and scheduling contention only involve interactions between each link and its conflicting neighbors. Therefore, each link $v = (i,j) \in \mathcal{V}$ only has to maintain its own utility $u(v,t)$ and the set of conflicting neighbors, denoted by $\mathcal{N}_{\mathcal{G}}(v)$. The implementation of link operations on transceivers is detailed in Section IV-B.

To minimize signaling overhead in the network, each link shall decide whether to contend for scheduling based on its own information, without talking with its neighbors. Hence, we want to find vertex-specific functions $\{h_v(\cdot)\}_{v \in \mathcal{V}}$ of local utility $u(v)$ to determine whether a vertex v should join the contention. Formally, we define our problem as follows.

Problem 1: Given a distribution Ω over network states $(\mathcal{G}, \mathbf{u})$ and a minimum required total utility $u_{\min}$, we want the optimal link sparsification functions $\{h_v^*\}$ for all $v \in \mathcal{V}$ as

$$\{h_v^*\} = \underset{\{h_v\}}{\operatorname{argmin}} \ \mathbb{E}_{\Omega}\left(|\mathcal{E}^s|\right) \tag{3a}$$

$$\text{s.t.} \ \ \mathcal{G}^s = \mathcal{G} \setminus \{v | v \in \mathcal{V}, h_v(u(v)) \leq 0\}, \tag{3b}$$

$$\mathcal{G}^s = (\mathcal{V}^s, \mathcal{E}^s), \tag{3c}$$

$$\mathbf{w}^s = [h_v(u(v))], \ \forall \ v \in \mathcal{V}^s, \tag{3d}$$

$$\hat{\boldsymbol{v}}^s = c(\mathcal{G}^s, \mathbf{w}^s), \tag{3e}$$

$$u_{\min} \leq \mathbb{E}_{\Omega}\left[u(\hat{\boldsymbol{v}}^s)\right]. \tag{3f}$$

To better understand Problem 1, first notice that constraints (3b) and (3c) define the sparsified conflict graph $\mathcal{G}^s = (\mathcal{V}^s, \mathcal{E}^s)$ by removing from the original conflict graph $\mathcal{G}$ those nodes (and, implicitly, all associated edges) with non-positive values of $h_v(u(v))$. Thus, it is immediate that the form of functions h_v has a direct influence on the level of sparsity of $\mathcal{G}^s$. Constraint (3d) defines a modified utility vector $\mathbf{w}^s$ for the nodes of the sparsified graph $\mathcal{G}^s$. Furthermore, constraint (3e) determines the schedule $\hat{\boldsymbol{v}}^s$ by applying a prescribed scheduler $c(\cdot)$ to $(\mathcal{G}^s, \mathbf{w}^s)$, and constraint (3f) specifies that the expected total utility of the schedule should be no less than a minimum value, which can be to set as the total utility of a schedule $\tilde{\boldsymbol{v}}^s$ under a baseline threshold policy, $u_{\min} = \mathbb{E}_{\Omega}\left[u(\tilde{\boldsymbol{v}}^s)\right]$, to take into account that the total utility is influenced by the size and density of networks. Our objective in (3a) is to minimize the expected number of edges in our sparsified graph, which determines the message complexity of the contention process. Finally, the inclusion of expectation in (3a) says that we do not want to find sparsification functions for a particular network state but rather functions that generalize well across a whole distribution of network states.

Finding an exact solution to Problem 1 is extremely challenging for several reasons including: i) The non-differentiable scheduler $c(\cdot)$ prevents direct application of gradient-based approaches, ii) The optimization is over the space of functions, which is infinite-dimensional, and iii) The sparsification functions should be valid for a distribution of network states, possibly associated with conflict graphs of different sizes and topologies. In the next section, we present our solution to Problem 1, which addresses these challenges.

IV. Link Sparsification With GNNs

In order to (approximately) solve Problem 1, we consider a parametric family of thresholding functions $\{h_v\}$ given by (2), in which $z(v) = \mathbf{z}_v$ is the link parameter employed by link $v \in \mathcal{V}$ to scale the prescribed global threshold, $u^{(\eta)}$, which is the η-quantile utility under the network state distribution Ω, as illustrated in Fig. 1(a). Notice that when $\mathbf{z} = \mathbf{1}$, the sparsification policy induced by $\{h_v\}$ in (2) boils down to the baseline policy in (1). Under the baseline, the Heaviside function compares the vertex utility $u(v)$ with $u^{(\eta)}$. If $u(v) \leq u^{(\eta)}$, then $h_v(u(v)) = 0$ and the node is excluded from the sparse graph $\mathcal{G}^s$ [cf. (3b)], otherwise, v is included in the sparse graph with the same utility $h_v(u(v)) = u(v)$. In this context, our proposed parameterization in (2) is a natural extension of the statistical baseline.

A. Generation of the Link Parameters

To ensure that functions $\{h_v(\cdot; \mathbf{z})\}$ can generalize to different underlying conflict topologies, we seek a function that can map a graph $\mathcal{G}$ to the vector of link parameters $\mathbf{z}$. Specifically, we propose to have $\mathbf{z} = \Psi_{\mathcal{G}}(\mathbf{1}; \boldsymbol{\omega})$, where $\Psi_{\mathcal{G}}$ is an L-layered graph convolutional network (GCN) defined on the conflict graph $\mathcal{G}$, and $\boldsymbol{\omega}$ is the collection of trainable parameters of the GCN. In a GCN layer, each node updates its features by applying a non-linear activation to the sum of two linear projections: one of its own features and one of the aggregated

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

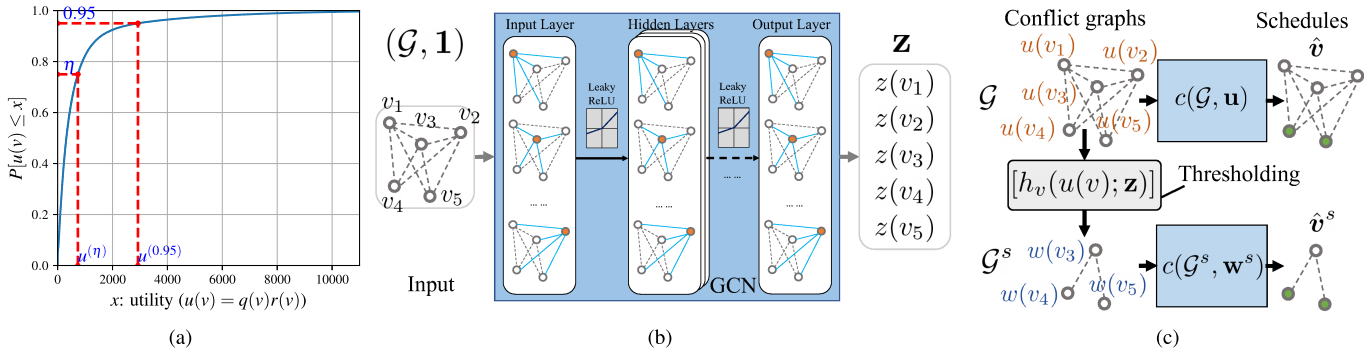


Fig. 1. Architecture of our GCN-based distributed sparse scheduler [1]. (a) Select $u^{(\eta)}$ based on the eCDF of utility. (b) Parameters $\mathbf{z}$ generated by a featureless GCN based on the conflict graph $\mathcal{G}$. (c) The sparse graph $\mathcal{G}^s$ and topology-aware utility $\mathbf{w}^s$ created by the parameterized functions $\{h_v(u(v); \mathbf{z})\}$ based on $(\mathcal{G}, \mathbf{u})$. The distributed sparse scheduler generates schedule as $\hat{\boldsymbol{v}}^s = c(\mathcal{G}^s, \mathbf{w}^s)$, whereas the dense scheduler generates schedules as $\hat{\boldsymbol{v}} = c(\mathcal{G}, \mathbf{u})$ for the purposes of training and comparison.

features from its immediate neighbors. These projections are based on trainable parameters, enabling an L-layered GCN to learn how to combine information from each node and its L-hop local neighbors for a particular task.

We define the output of an intermediate lth layer of the GCN as $\mathbf{X}^l \in \mathbb{R}^{|\mathcal{V}| \times g_l}$, and $\mathbf{X}^0 = \mathbf{1}^{|\mathcal{V}| \times 1}$, $\mathbf{z} = \mathbf{X}^L$, then the lth layer of the GCN is expressed as:

$$\mathbf{X}^l = \sigma_l \left(\mathbf{X}^{l-1} \boldsymbol{\Theta}_0^l + \boldsymbol{\mathcal{L}} \mathbf{X}^{l-1} \boldsymbol{\Theta}_1^l \right), l \in \{1, \ldots, L\}. \quad (4)$$

In (4), $\boldsymbol{\Theta}_0^l, \boldsymbol{\Theta}_1^l \in \mathbb{R}^{g_{l-1} \times g_l}$ are trainable parameters, $\sigma_l(\cdot)$ is the element-wise activation function of the lth layer, and $\boldsymbol{\mathcal{L}}$ is the normalized Laplacian of $\mathcal{G}$, i.e., $\boldsymbol{\mathcal{L}} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$, where $\mathbf{I}$ is the identity matrix, $\mathbf{A}$ is the adjacency matrix of $\mathcal{G}$, and $\mathbf{D}^{-1/2} = \text{diag}([1/\sqrt{d(v)} \mid v \in \mathcal{V}])$ where $d(v)$ is the degree of vertex v. The activation functions of the input and hidden layers are selected as leaky ReLUs whereas a ReLU activation is used for the output layer. The input and output dimensions are set as $g_0 = g_L = 1$.

As a local operator on $\mathcal{G}$, the normalized Laplacian $\boldsymbol{\mathcal{L}}$ in (4) allows $z(v)$ to be computed in a distributed manner through neighborhood aggregation at v with L rounds of local exchanges between v and its neighbors. Specifically, (4) can be implemented by the following local operation on link $v \in \mathcal{V}$,

$$\mathbf{X}_{v*}^l = \sigma_l \left(\mathbf{X}_{v*}^{l-1} \boldsymbol{\Theta}_0^l + \left[\mathbf{X}_{v*}^{l-1} - \sum_{u \in \mathcal{N}_{\mathcal{G}}(v)} \frac{\mathbf{X}_{u*}^{l-1}}{\sqrt{d(v) d(u)}} \right] \boldsymbol{\Theta}_1^l \right), \quad (5)$$

where $\mathbf{X}_{v*}^l \in \mathbb{R}^{1 \times g_l}$ is the vth row of matrix $\mathbf{X}^l$, which captures the features on vertex v, $d(v)$ is the degree of vertex v, and $\mathcal{N}_{\mathcal{G}}(v)$ is the set of neighboring vertices of v on $\mathcal{G}$.

The downstream architecture of the entire distributed sparse scheduler [1] is illustrated in Fig. 1. First, the prior knowledge of eCDF of the utility values from empirical data of network operations is collected, as shown in Fig. 1(a), based on which a global threshold is selected as $u^{(\eta)}$. In practice, this eCDF can be collected by a subset of devices based on the link utilities of themselves and their neighbors. Next, at the network level, the trained GCN observes the topology of the network and generates node embeddings as the link parameters $\mathbf{z}$. Then, the sparse graph $\mathcal{G}^s$ and topology-aware utility vector $\mathbf{w}^s$ are obtained by the parameterized functions $\{h_v(u(v); \mathbf{z})\}$ according to (3b) and (3d), respectively. Finally, the sparse schedule is obtained by the distributed scheduler as $\hat{\boldsymbol{v}}^s = c(\mathcal{G}^s, \mathbf{w}^s)$ with lowered scheduling overhead.

We define local communication complexity as the rounds of local exchanges between a node and its neighbors. The local communication complexity of a GCN is $\mathcal{O}(L)$. Although each round of local exchange in GCN comprises $2|\mathcal{V}||\mathcal{E}|$ point-to-point messages, they can be implemented as $|\mathcal{X}|$ broadcast transmissions in wireless networks (see Section IV-B). Hence, the local computational and communication costs can be controlled by modifying the number of layers L in the GCN. Importantly, this constant local communication complexity is critical to scalability. Moreover, the link parameters $\mathbf{z}$ can be reused across time slots and only need to be updated when the topology changes, causing negligible impacts to wireless networks with relatively low mobility by diluting the overhead of generating $\mathbf{z}$ into hundreds to thousands of time slots.

B. Transceiver-Level Operations for Directed Links

Next, we discuss transceiver-level operations in the entire sparse scheduling pipeline, such as link maintenance (including conflict graph construction and hidden node problem avoidance), GCN operations, and scheduling contention. We focus on the case of directed links as it is more relevant to massive access. The distributed operations of GCN in (5) and contention $c(\cdot)$ only require a link v to know its immediate conflicting neighbors $\mathcal{N}_{\mathcal{G}}(v)$ rather than the entire conflict graph $\mathcal{G}$. Therefore, the construction of $\mathcal{G}$ is equivalent to each link $v \in \mathcal{V}$ collecting and maintaining $\mathcal{N}_{\mathcal{G}}(v)$, which can be blended in the processes of initial access and recurrent contentions. Conceptually [12, supplemental materials], we let the control messages of each link v always carry its identity v, degree $d(v)$, and feature $\mathbf{X}_{v*}^l$, which enables a link to collect the IDs of its conflicting neighbors $\mathcal{N}_{\mathcal{G}}(v)$, update $d(v)$, and implement GCN layers in (5) by monitoring the channel over a sliding window of sufficient duration.

When a new device joins the network, it acquires a subchannel for sending control messages through initial access on a dedicated random access channel, e.g., by defective graph coloring on $\mathcal{G}$ with a local complexity of $\mathcal{O}(1)$ [56]. The transmitter of a link maintains its utility and performs sparsification based on its updated backlog information, whereas the receiver

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Local Greedy Solver (LGS) for Directed Links

Input: $\mathcal{X}, \mathcal{V}, \mathcal{E}, \mathcal{G}^s = (\mathcal{V}^s, \mathcal{E}^s), \mathbf{u} \in \mathbb{R}_+^{|\mathcal{V}|}$
Output: $\mathbf{v} \in \{0,1\}^{|\mathcal{V}|}$ /* *0: mute, 1: scheduled* */
$\mathbf{v} = -\mathbf{1} + \mathbf{1}_{\{\mathcal{V} \notin \mathcal{V}^s\}}$, $\mathcal{G}^n = (\mathcal{X}, \mathcal{V}), \mathcal{G} = (\mathcal{V}, \mathcal{E})$
while $\mathbf{v}_v \neq -1, \forall v \in \mathcal{V}$ **do**
  for all $i \in \mathcal{X}$ **do**
    $\mathcal{T}_i = \{(i,j) | \mathbf{v}_{(i,j)} < 0, j \in \mathcal{N}_{\mathcal{G}^n}(i)\}$, $d_i^o = |\mathcal{N}_{\mathcal{G}^n}(i)|$
    if $\mathcal{T}_i \neq \emptyset$ **then**
      $v_i = \text{argmax}_{v \in \mathcal{T}_i} \mathbf{u}_v$ /* *Internal contention* /*
      $\mathbf{v}_v = \mathbb{1}(v = v_i),\ \forall v \in \mathcal{T}_i$
      Broadcast a RTS message $\{i, d_i^o, \kappa_i, v_i, \mathbf{u}_{v_i}\}$
    end if
    Receive RTS messages from nearby devices, denoted as set $\tilde{\mathcal{N}}_{\mathcal{G}}(i) = \big\{j \mid \exists x_1, x_2,\ ((j,x_1),(x_2,i)) \in \mathcal{E}\big\}$
    $\Upsilon_i = \bigcup_{j \in \tilde{\mathcal{N}}_{\mathcal{G}}(i) \cup \{i\}} \{v_j\}$, $d_i^c = \sum_{j \in \tilde{\mathcal{N}}_{\mathcal{G}}(i) \cup \{i\}} d_j^o$
    $\tilde{v}_i = \text{argmax}_{v \in \Upsilon_i} \mathbf{u}_v$ /* *OTA contention* /*
    if i is the receiver of link $\tilde{v}_i$ **then**
      $\mathbf{v}_{\tilde{v}_i} = 1$, broadcast a CTS message with $(\tilde{v}_i, d_i^c)$
    else if Receive CTS message $(\tilde{v}_j, d_j^c)$ **then**
      $\mathbf{v}_{\tilde{v}_j} = \mathbb{1}(i \text{ is the transmitter of link } \tilde{v}_j)$
    end if
  end for
end while

of link v maintains its conflicting neighbors $\mathcal{N}_{\mathcal{G}}(v)$. Each device $i \in \mathcal{X}$ first decides a winner v_i of internal contention among its undecided outgoing links, then it broadcasts a request-to-send (RTS) message, e.g., $\{i, d_i^o, \kappa_i, v_i, u(v_i)\}$, with its identity i, outgoing degree d_i^o and outgoing link set κ_i, the identity and utility of link v_i. Every device also listens to the control channel to collect RTS messages from nearby devices. All the unique links detected by a device (with sufficient receive signal strength) over a sliding window, together with its own outgoing links, are in conflict with each other. A device computes the GCN in (5) and resolves OTA contentions for all of its incoming links. When a directed link (i,j) wins the OTA contention, its receiver j broadcasts a clear-to-send (CTS) message $\{(i,j), d_j^c\}$ to notify the sender i and mute its conflicting links $\mathcal{N}_{\mathcal{G}}((i,j))$, avoiding the hidden node problem. Here, d_j^c is the number of conflicting links sensed by device j. A transmitter hearing this CTS learns that the conflict degrees of its outgoing links are at least $d_j^c - 1$.

The aforementioned transceiver-level principle can be translated into a distributed MaxWeight scheduler [17] in Algorithm 1 and CSMA-based protocols [23], [24], [25]. For CSMA, the contention is randomized and could be influenced by link utilities [23], in particular, the OTA contention for RTS messages is based on timing, where the first RTS picked up by a receiver wins. When a device $j \in \mathcal{X}$ finds that the first RTS it received is addressed to itself, it broadcasts a CTS message to notify the sender and mute other nearby transmitters.

C. Asynchronous Implementation of GCNs

Since link parameters $\mathbf{z}$ are decoupled from the real-time link utilities $\mathbf{u}(t)$, an L-layer GCN can be computed asynchronously, e.g., when a new link joins the network, only its L-hop conflicting neighbors need to update their link parameters, for which the message exchanges can take many time slots by enclosing $\{\mathbf{X}_{v*}^l\}$ in the RTS and CTS for contentions. In the distributed execution of a GCN layer, a device i first exchanges a control message with its nearby devices, which encloses the features of their outgoing links, e.g., $\{\mathbf{X}_{v*}^{l-1}\}_{v \in \kappa_i}$. From the received control messages, the device sorts out the features of its incoming links and their conflicting neighbors, then computes (5) for each of its own incoming links, and exchanges the results $\{\mathbf{X}_{v*}^l\}_{v \in \rho_i}$ with nearby devices to obtain the output features of its outgoing links, $\{\mathbf{X}_{v*}^l\}_{v \in \kappa_i}$.

The most practical option is a 1-layer GCN with $L = 1, g_0 = g_1 = 1$, not only because of its effective yet lightweight model with just two trainable parameters, but also its friendliness to asynchronous implementation. Specifically, by enclosing $\{\mathbf{X}_{v*}^{l-1}\}_{v \in \kappa_i}$ in the RTS and $\{\mathbf{X}_{v*}^l\}_{v \in \rho_i}$ in the CTS for all devices $i \in \mathcal{X}$, a newly joined device j can learn the conflicting neighbors of its incoming links by passively monitoring the channel for a sufficiently long period, and get the link parameters $\{\mathbf{z}_v\}_{v \in \kappa_j}$ for its outgoing links with only one RTS-CTS cycle. In networks with low mobility, such a lightweight, asynchronous 1-layer GCN can minimize the overhead of establishing the local threshold policies.

D. Global Threshold

The optimal global threshold $u^{(\eta^*)}$ is typically set manually based on prior knowledge of the wireless networks. Ideally, the optimal cut-off quantile η^* shall maximize the expected total utility net of signaling overhead, defined as follows

$$\eta^* = \underset{0<\eta\leq 1}{\text{argmax}}\ \mathbb{E}_{\Omega}\left\{ \sum_{v \in \bar{\boldsymbol{v}}(\mathcal{G}^s)} u(v)\big[1 - f_s\big(d^s(v)+1, \tau\big)\big] \middle| \eta \right\}, \quad (6)$$

where $0 < \tau \ll 1$ is the fraction of time slot (frame) for a link to broadcast a control message to its conflicting neighbors, $f_s(x,\tau)$ encodes that the scheduling overhead is a function of τ and the neighborhood size x, $d^s(v)$ is the expected degree of node v on the sparsified graph $\mathcal{G}^s$ if it is not cutoff, Ω is the distribution of network states $(\mathcal{G}, \mathbf{u})$, and $\bar{\boldsymbol{v}}(\mathcal{G}^s)$ is a maximal independent set on $\mathcal{G}^s$ that contains the schedule, $\hat{\boldsymbol{v}}^s \subseteq \bar{\boldsymbol{v}}(\mathcal{G}^s)$.

For local greedy solver (LGS)-based protocols [12], [17], [22], the first round of message exchange dominates the entire point-to-point message complexity. If a sender started transmission upon receiving its CTS, denoted as *flexible overhead*, the overhead can be approximated as $f_s(x,\tau) \approx \min(1, x\tau)$, otherwise, if all transmissions began only after a fixed scheduling deadline of $K\tau$, the overhead $f_s(x,\tau) \approx \mathbb{1}(x\tau > K\tau)$, representing a link $v \in \bar{\boldsymbol{v}}(\mathcal{G}^s)$ is not scheduled due to signaling overdue. In both cases, $f_s(x)$ monotonically increases with x. For CSMA-based protocols with a fixed contention or backoff window W, the objective in (6) can be rewritten as

$$\mathbb{E}_{\Omega}\left\{ \frac{1}{W} \sum_{v \in \mathcal{V}^s} u(v) \sum_{m=0}^{W-1} \left(\frac{W-1-m}{W} \right)^{d^s(v)} \middle| \eta \right\}.$$

The approximated forms of $f_s(x)$ enable quick evaluation of the objective function in (6) for a given η by sampling the

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

graph $\mathcal{G} \in \Omega$, the link utility $u(i)$, and the maximal independent set $\bar{\boldsymbol{v}}(\mathcal{G}^s)$, which empirically exhibits quasi-concavity as shown in Fig. 4(c). Therefore, the optimal η^* can be found by an efficient search algorithm described in Appendix B. In practice, $u^{(\eta)}$ can be refined periodically based on network dynamics.

E. Hybrid Threshold Policy

In practice, our GCN-based threshold policy can be conditionally applied to a link when the configuration of wireless protocols does not support its conflict degree:

$$h_v(u(v); z) = u(v)\, H\left(u(v) - \mathbb{1}(d(v) > D) z(v) u^{(\eta)}\right), \quad (7)$$

where D is set to ensure that the overhead of the first round of local exchange $D\tau$ is within the scheduling sub-slot, e.g., $D < K$. As shown in Section VI-C, this hybrid policy can avoid unnecessary delays in less crowded parts of the network.

V. First Order Optimization

Having discussed the downstream pipeline of the threshold-based link sparsification, we now turn to constrained unsupervised learning (UL) for training the GCNs. Challenges stem from two aspects: 1) the GCN is followed by non-differentiable operations, $\{h_v(\cdot)\}$ and $c(\cdot)$, and 2) we have to balance two competing objectives: minimizing the overhead while achieving a minimum level of total utility. To address these challenges, we reformulate Problem 1 as Problem 2 by introducing GCN-based parameterization of $\{h_v(\cdot)\}$, setting the minimum total utility to the statistical baseline, $u_{\min} = \mathbb{E}_\Omega[u(\tilde{\boldsymbol{v}}^s)]$, and bridging the non-differentiability with surrogate gradient in (12)–(14).

Problem 2: Given a distribution Ω over network states $(\mathcal{G}, \mathbf{u})$, where the utilities $u(v) \geq 0$ for $v \in \mathcal{V}$ follow an i.i.d. distribution Ω_u, we want to obtain the optimal set of parameters, $\boldsymbol{\omega}$, for the GCN $\mathbf{z} = \Psi_{\mathcal{G}}(\mathbf{1}; \boldsymbol{\omega})$, where $\mathbf{z}_v \geq 0\ \forall\, v \in \mathcal{V}$, as

$$\boldsymbol{\omega}^* = \underset{\boldsymbol{\omega} \in \mathbb{R}^{|\boldsymbol{\omega}|}}{\operatorname{argmin}} \ \mathbb{E}_{(\mathcal{G},\mathbf{u})\sim\Omega}\left(|\mathcal{E}^s|\right) \quad (8a)$$

$$\text{s.t.} \quad \mathcal{G}^s = \mathcal{G} \setminus \{v | v \in \mathcal{V}, h_v(u(v); z) \leq 0\}, \quad (8b)$$

$$\tilde{\mathcal{G}}^s = \mathcal{G} \setminus \{v | v \in \mathcal{V}, h_v(u(v); \mathbf{1}) \leq 0\}, \quad (8c)$$

$$\mathbf{w}^s = [h_v(u(v); z)],\ \forall\, v \in \mathcal{V}^s, \quad (8d)$$

$$\tilde{\mathbf{w}}^s = [h_v(u(v); \mathbf{1})],\ \forall\, v \in \tilde{\mathcal{V}}^s, \quad (8e)$$

$$\tilde{\boldsymbol{v}}^s = c(\tilde{\mathcal{G}}^s, \tilde{\mathbf{w}}^s), \quad (8f)$$

$$\hat{\boldsymbol{v}}^s = c(\mathcal{G}^s, \mathbf{w}^s), \quad (8g)$$

$$0 \leq \mathbb{E}_\Omega\left[u(\hat{\boldsymbol{v}}^s) - u(\tilde{\boldsymbol{v}}^s) | (\mathcal{G}, \mathbf{u})\right], \quad (8h)$$

$$z = \Psi_{\mathcal{G}}(\mathbf{1}; \boldsymbol{\omega})\ . \quad (8i)$$

The constraints in (8b)–(8g) specify the respective processes of link sparsification under our approach and the statistical baseline. The constraint in (8h) sets the expected total utility of our approach to be no less than that of the statistical baseline.

On an arbitrary graph $\mathcal{G}$ with utility vector $\mathbf{u} \sim \Omega_{\mathbf{u}}^{\mathcal{G}}$, the objective in (8a) has an analytical form

$$\mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left(|\mathcal{E}^s| \middle| \mathcal{G}\right) = \frac{1}{2} \sum_{v \in \mathcal{V}} d^s(v)\left[1 - p(v)\right]\ , \quad (9)$$

Algorithm 2 Alt-SGD: Alternating SGD for Constrained UL

Input: $\Omega, f_u(\cdot), \alpha, 0 < \beta \leq 1, N$, BatchSize
Output: $\boldsymbol{\omega}^*$

1: Initialize $\boldsymbol{\omega}$ randomly, $\mathcal{Q} \leftarrow \varnothing$
2: **for** epoch **do**
3: Draw $\eta \sim [0, 1)$, $(\mathcal{G}, \mathbf{u}) \sim \Omega$
4: Compute $\mathbf{z}$ based on (8i), $u^{(\eta)} = f_u^{-1}(\eta)$.
5: Compute $u(\hat{\boldsymbol{v}}^s), u(\tilde{\boldsymbol{v}}^s)$ using (8b)–(8g).
6: **if** $u(\hat{\boldsymbol{v}}^s) < u(\tilde{\boldsymbol{v}}^s)$ **then**
7: $\widehat{\nabla_{\mathbf{z}} J(\boldsymbol{\omega})} = -\nabla_{\mathbf{z}} \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[u(\hat{\boldsymbol{v}}^s) | \mathcal{G}\right]$ based on (14), (13)
8: **else**
9: $\widehat{\nabla_{\mathbf{z}} J(\boldsymbol{\omega})} = \nabla_{\mathbf{z}} \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left(|\mathcal{E}^s| \middle| \mathcal{G}\right)$ based on (11)
10: **end if**
11: $\widehat{\nabla_{\boldsymbol{\omega}} J(\boldsymbol{\omega})} = \nabla_{\boldsymbol{\omega}} \Psi_{\mathcal{G}}(\mathbf{1}, \boldsymbol{\omega}) \widehat{\nabla_{\mathbf{z}} J(\boldsymbol{\omega})}$
12: $\widehat{\nabla_{\boldsymbol{\omega}} J(\boldsymbol{\omega})} \leftarrow N \widehat{\nabla_{\boldsymbol{\omega}} J(\boldsymbol{\omega})} / \left\| \widehat{\nabla_{\boldsymbol{\omega}} J(\boldsymbol{\omega})} \right\|_2$
13: $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\widehat{\nabla_{\boldsymbol{\omega}} J(\boldsymbol{\omega})}\}$
14: **if** $|\mathcal{Q}| \geq$ BatchSize **then**
15: **for** $\widehat{\nabla_{\boldsymbol{\omega}} J(\boldsymbol{\omega})} \in \mathcal{Q}$ **do**
16: $\boldsymbol{\omega} \leftarrow \boldsymbol{\omega} - \alpha \widehat{\nabla_{\boldsymbol{\omega}} J(\boldsymbol{\omega})}$
17: **end for**
18: $\mathcal{Q} \leftarrow \varnothing$, $\alpha \leftarrow \alpha\beta$
19: **end if**
20: **end for**

where $d^s(v)$ can be computed as follows

$$d^s(v) = d(v) - \sum_{i \in \mathcal{N}_{\mathcal{G}}(v)} p(i). \quad (10)$$

In (10), $p(v) = f_u\left(z(v) u^{(\eta)}\right)$ is the cutoff probability of vertex v, where $f_u(\cdot)$ is the eCDF of per-link utility. Based on (9), (10), the probability density function (PDF) of link utility $f_u'(\cdot)$, and the derivation in Appendix A, we have

$$\frac{\partial \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left(|\mathcal{E}^s| \middle| \mathcal{G}\right)}{\partial z(v)} = -d^s(v) f_u'\left(z(v) u^{(\eta)}\right) u^{(\eta)}\ . \quad (11)$$

We propose to solve (8) through alternating stochastic gradient descent (Alt-SGD), as described in Algorithm 2, which minimizes the objective in (8a) while maintaining the inequality in (8h). More precisely, when (8h) is satisfied, we update $\boldsymbol{\omega}$ (in line 9) in the direction of the negative gradient $\nabla_{\boldsymbol{\omega}} \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left(|\mathcal{E}^s| \middle| \mathcal{G}\right) = \nabla_{\boldsymbol{\omega}} \Psi_{\mathcal{G}}(\mathbf{1}, \boldsymbol{\omega}) \nabla_{\mathbf{z}} \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left(|\mathcal{E}^s| \middle| \mathcal{G}\right)$. Otherwise, we enforce satisfaction of (8h) (in line 7) by updating $\boldsymbol{\omega}$ in the direction of the gradient $\nabla_{\boldsymbol{\omega}} \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[u(\hat{\boldsymbol{v}}^s) \middle| \mathcal{G}\right] = \nabla_{\boldsymbol{\omega}} \Psi_{\mathcal{G}}(\mathbf{1}, \boldsymbol{\omega}) \nabla_{\mathbf{z}} \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[u(\hat{\boldsymbol{v}}^s) \middle| \mathcal{G}\right]$, to increase utility, where

$$\mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[u(\hat{\boldsymbol{v}}^s) \middle| \mathcal{G}\right] = \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[\mathbf{u}^\top \hat{\mathbf{v}}^s \middle| \mathcal{G}\right] = \sum_{v \in \mathcal{V}} \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[\mathbf{u}_v \hat{\mathbf{v}}_v^s \middle| \mathcal{G}\right], \quad (12)$$

and $\hat{\mathbf{v}}^s \in \{0, 1\}^{|\mathcal{V}|}$ is the indicator vector of set $\hat{\boldsymbol{v}}^s$ w.r.t. $\mathcal{V}$. Since $\mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[\mathbf{u}_v \hat{\mathbf{v}}_v^s \middle| \mathcal{G}\right]$ as a function of $p(v)$ does not have any analytical form, we consider two analytical proxies:

$$\mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[\mathbf{u}_v \hat{\mathbf{v}}_v^s \middle| \mathcal{G}\right] \approx a_1\left[1 - p(v)\right]\ , \quad (13a)$$

$$\mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[\mathbf{u}_v \hat{\mathbf{v}}_v^s \middle| \mathcal{G}\right] \approx a_2\left[1 - p(v)\right]\left[1 - a_3 d^s(v)\right]\ , \quad (13b)$$

where $a_1, a_2 > 0$ and $0 < a_3 < 1$ are constant parameters. The rationale behind (13a) is straightforward: the more likely

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

a link $v \in \mathcal{V}$ is retained after sparsification, the larger is the expected utility gained from v. The additional term $1-a_3 d^s(v)$ in (13b) further captures the secondary factor that the expected utility gained on link v is reduced if there are more conflicting neighbors of link v after sparsification. Based on (12) and (13), we can find the gradient $\nabla_{\mathbf{z}} \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[u(\hat{\boldsymbol{v}}^s)\middle|\mathcal{G}\right]$ as:

$$\frac{\partial \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[u(\hat{\boldsymbol{v}}^s)\middle|\mathcal{G}\right]}{\partial z(v)} = \frac{\partial \mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left[\mathbf{u}_v \hat{\mathbf{v}}_v^s\middle|\mathcal{G}\right]}{\partial p(v)} f_u'\Big(z(v)u^{(\eta)}\Big)u^{(\eta)}. \quad (14)$$

Since $f_u(\cdot)$ does not have an analytical form, we use a multilayer perceptron (MLP) $f_u(\cdot;\boldsymbol{\theta})$ to fit $f_u(\cdot)$ in order to approximately compute the gradients in (11) and (14).

Problem 2 is non-convex due to the integer nature of the objective and constraints in (8). Although the convergence of Algorithm 2 is not theoretically guaranteed, its intuition is as follows. The gradient in line 7 pushes the policy back into the constrained region of utility whenever it falls outside of it, whereas the gradient in line 9 moves the policy towards lower cost. With global norm being clipped in line 12, these two types of gradients share the same step size, but may not be in completely opposite directions. Therefore, the aggregation of sample gradients over a mini batch would likely point the policy to a direction of Pareto improvement, unless it is already in a locally Pareto optimal area within the constrained region, which is typically superior to the baseline policy. By using a decaying learning rate (line 18), we gradually reduce the step size so that the policy can always converge after a prescribed number of epochs, which is our stop criterion.

VI. Numerical Experiments

We evaluate the GCN-based threshold policy, $h_v(u(v);\mathbf{z})$, as a component of sparse schedulers in simulated time-slotted networks with synthetic random conflict graphs, which seek to represent wireless networks with uniformly distributed users of identical omnidirectional transmit power. Three GCNs with number of layers $L \in \{1,2,3\}$, denoted by GCN(L), are evaluated, with an emphasis on GCN(1). These policies are compared against the statistical baseline with a global threshold, $h_v(u(v);\mathbf{1})$, as described in the beginning of Section IV, and the zero-threshold policy, $h_v(u(v);\mathbf{0})$ (dense scheduler). These threshold policies are tested on four contention functions $c(\cdot)$, based on MaxWeight (LGS [17]) and CSMA [23].

A. Experimental Configurations

The training settings include a learning rate of $\alpha = 10^{-4}$, $\beta = 0.996$, $N = 0.05$, a batch size of 100, and a total of 25 epochs.[1] The network states $(\mathcal{G},\mathbf{u})$ for training are generated as follows: The link utility $u(v)=\mathbf{u}_v, v \in \mathcal{V}$ is independently drawn from an empirical distribution (see Figs. 1(a) and 4(b)), which is collected from simulation of a specific scheduling protocol on a set of synthetic conflict graphs and the following network traffic configurations.

[1]Training takes 10-20 hours on a workstation with a specification of 16GB memory, 8 cores, and Geforce GTX 1070 GPU. The source code is published at https://github.com/zhongyuanzhao/gcn-sparsify

Network traffic: The link utility function is $u(v,t) = q(v,t)r(v,t)$ [17], and the backlog (queue length) evolves as

$$q(v,t+1)=q(v,t)-\mathbf{v}_v(t)\min\{r(v,t),q(v,t)\}+a(v,t),$$

where $a(v,t)$ is the number of exogenous packets arriving at the source of link v at time step t, $\mathbf{v}_v(t) = 1$ (or 0) if link v is (not) scheduled at time t. The nominal link rate $r(v,t)$ for each link and time step is independently drawn from a normal distribution $\mathbb{N}(50,25)$, rounded up, and then clipped to $[0,100]$, which captures the effects of fading and lognormal shadowing in wireless channel [57]. For simplicity, the data traffic is configured as a single hop flow generated for each link $v \in \mathcal{V}$. For each flow, the exogenous packets at the source user follow a Poisson arrival with a prescribed arrival rate λ. The traffic load, defined as $\mu = \lambda/\mathbb{E}_{v\in\mathcal{V},t\le T}[r(v,t)]$, is set as $\mu \in [0.03, 0.05]$ for all threshold policies, offering unsaturated traffic for sparser wireless networks, i.e., the average conflict degree $\bar{d} \le 25$. Each scheduling instance includes a conflict graph $\mathcal{G} = (\mathcal{V},\mathcal{E})$, and realizations of random packet arrivals $a(v,t)$ and link rates $r(v,t)$ for $v \in \mathcal{V}, 0 < t \le T$ where $T = 200$. Although each scheduling instance has a static topology, we test a trained GCN on many conflict graphs with different sizes and topologies without any re-training, therefore, the GCN-based threshold policies can generalize to dynamic topology due to network mobility.

Conflict graph: Four datesets of synthetic conflict graph $\mathcal{G}$ for training and testing are generated from the Erdős–Rényi (ER) [58] model and Barabási–Albert (BA) [59, Ch 5] model, as detailed in Table I, where $|\mathcal{V}|$ is the size of graph, k is the probability of edge-appearance in ER model, m is the preferential attachment parameter in BA model, and $\bar{d}$ is the average degree of a graph. For BA graphs, $\bar{d} = 2m$ [59, Ch 5]. For each experiment, the training and test sets may or may not be from the same graph model, as shown in Table I. Notice the slight mismatch between training and test sets, i.e., $\bar{d} = 15, 20$ ($m \ge 15$, $|\mathcal{V}| > 300$) are not in the ER (BA) training set.

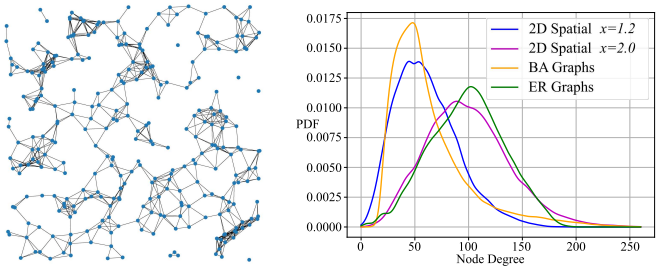
To illustrate how the ER and BA models can capture the conflict graphs, we compare the degree distributions of the ER and BA test sets with the conflict graphs of 100 instances of wireless multi-hop networks. Each network instance is generated from a 2D spatial process, in which 300 wireless devices, equipped with uniform transmit power and omnidirectional antennas, are randomly located in a 12×12 square; a link (edge) is formed if two devices (nodes) are within a distance of 1.0; and two links interfere with each other if any of their incidental nodes are within a distance of $x \in \{1.2, 2.0\}$. For generality, the unit of distance is omitted since path loss is a function of both carrier frequency and distance. The nodes and links in one of these instances are visualized in Fig. 2(a). The PDFs of conflict degree of these network instances are presented in Fig. 2(b). With smaller interference radius ($x = 1.2$), the distribution of node degrees of the conflict graphs is right-skewed, matching the degree distribution of graphs in the BA test set. With larger interference radius ($x = 2.0$), the conflict degree distribution is more symmetric and similar to a set of ER graphs (not in Table I) with $|\mathcal{V}| \in \lfloor \mathbb{U}(200,400) \rceil$ and $\bar{d} \in \mathbb{N}(100,30)$.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
CONFIGURATIONS FOR DATASETS OF SYNTHETIC RANDOM GRAPHS

Set (size)	Parameters	Quantity	Total
ER training set (5900)	$\lvert\mathcal{V}\rvert \in \{100, 150, 200, 250, 300\}$, $\bar{d} = \lvert\mathcal{V}\rvert k \in \{2, 5, 7.5, 10, 12.5\}$	200 instances per $(\lvert\mathcal{V}\rvert, \bar{d})$	5000
	$\lvert\mathcal{V}\rvert \in \{30, 100\}$, $k \in \{0.1, 0.2, \ldots, 0.9\}$	50 instances per $(\lvert\mathcal{V}\rvert, k)$	900
BA training set (5900)	$\lvert\mathcal{V}\rvert \in \{100, 150, 200, 250, 300\}$, $m \in \{2, 5, 7.5, 10, 12.5\}$	200 instances per $(\lvert\mathcal{V}\rvert, m)$	5000
	$\lvert\mathcal{V}\rvert \in \{30, 100\}$, $m = \lvert\mathcal{V}\rvert k$, $k \in \{0.1, 0.2, \ldots, 0.9\}$	50 instances per $(\lvert\mathcal{V}\rvert, k)$	900
ER test set (500)	$\lvert\mathcal{V}\rvert \in \{100, 150, 200, 250, 300\}$, $\bar{d} = \lvert\mathcal{V}\rvert k \in \{2, 5, 10, 15, 20\}$	20 instances per $(\lvert\mathcal{V}\rvert, \bar{d})$	500
BA test set (860)	$\lvert\mathcal{V}\rvert \in \{100, 150, 200, 250, 300\}$, $m \in \{2, 5, 10, 15, 20\}$	20 instances per $(\lvert\mathcal{V}\rvert, m)$	500
	$\lvert\mathcal{V}\rvert \in \{300, 400, 500\}$, $m \in \{25, 30, 35, 40, 45, 50\}$	20 instances per $(\lvert\mathcal{V}\rvert, m)$	360

Section	Experiments	Training set	Test set
VI-B	Total utility with identical inputs	ER-train	ER-test
VI-C1	Latency with LGS (fixed deadline)	ER-train	BA-test
VI-C2	Latency with LGS (flexible overhead)	BA-train	BA-test
VI-C3	Latency with two CSMA schedulers	ER-train	BA-test



(a) (b)

Fig. 2. Random instances of wireless multi-hop networks generated from a 2D spatial process, with two ratios of interference to connectivity radii, $x \in \{1.2, 2.0\}$: a) An instance of wireless networks, where an edge represents a link. b) The degree distributions of 100 instances of conflict graphs with $x = 1.2$ and $x = 2.0$, the BA test set in Table I, and another set of ER graphs (not listed in Table I) with $\lvert\mathcal{V}\rvert \in \lfloor\mathbb{U}(200, 400)\rceil$ and $\bar{d} \in \mathbb{N}(100, 30)$.

B. Overhead Under Identical Network State

We first compare the overheads of various threshold policies under LGS schedulers per time-slot, given identical network states and cut-off quantiles. For an input network state $(\mathcal{G}(i), \mathbf{u}(i))$, the conflict graph $\mathcal{G}(i)$ is drawn from the ER test set, and the link utility $\mathbf{u}_v(i), v \in \mathcal{V}$ is drawn from the empirical distribution of per-link utility (Fig. 1(a)) collected from an ideal LGS scheduler with realized link rate equals to the nominal one, under the network traffic setting and ER test set described in Section VI-A. Each network state is tested with 12 cut-off quantiles $\eta \in \{0, 0.1, \ldots, 0.8, 0.85, 0.9, 0.95\}$.

Besides the regular GCN(L) with $L \in \{1, 2, 3\}$ trained with Alt-SGD, we also train a GCN(1) with ZOO to serve as the baseline for Alt-SGD. Additionally, to evaluate the benefit of selecting GCN as the ML model, we train another 5-layer perceptron with Alt-SGD, denoted by MLP(5), which resides on each node and takes the degree of its own node as the only input – without the neighborhood aggregation in GNNs. Notice that this MLP(5) is different from the MLP $f_u(\cdot; \boldsymbol{\theta})$ used in (11) and (14) for training.

TABLE II
DESCRIPTION OF MAJOR NOTATIONS

Symbols	Descriptions
α, β	α: learning rate, β: decaying rate of α
η	global cut-off quantile $0 \le \eta \le 1$
τ	fraction of a time slot or frame duration for a link to broadcast a control message to its interfering neighbors
$\boldsymbol{\Theta}_0^l, \boldsymbol{\Theta}_1^l$	sets of trainable parameters of layer l of the GCN in (4)
λ, μ	λ arrival rate, $\mu = \mathbb{E}(r)/\lambda$: traffic load
κ_i, ρ_i	κ_i (ρ_i) set of outgoing (incoming) links for device i
$\sigma_l(\cdot)$	element-wise activation function for layer l of GCN
$\boldsymbol{\Psi}_{\mathcal{G}}(\cdot; \boldsymbol{\omega})$	GCN defined on graph $\mathcal{G}$ with trainable parameters $\boldsymbol{\omega}$
Ω	distribution of network state $(\mathcal{G}, \mathbf{u})$
$a(v, t)$	No. of exogenous packets arriving link v in time slot t
$c(\cdot)$	function of the contention process
$d(v), d^s(v)$ $\bar{d}$	$d(v)$: degree of v, $d^s(v)$: post-sparsification degree of v, $\bar{d}$ average vertex degree of graph $\mathcal{G}$
D	minimal conflict degree to enable threshold in hybrid policy
$f_c(\cdot)$ $f_s(\cdot)$	function of collision rate function of signaling overhead
$f_u(\cdot), f_u'(\cdot)$	the CDF and PDF of per-link utility
$f_{\mathbb{E}(u)}(\cdot)$	function of the expected utility w.r.t. the cut-off probability for an individual link
g_l	dimension of output features of layer l of GCN
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	conflict graph $\mathcal{G}$, composed of a set of vertices $\mathcal{V}$ and a set of edges $\mathcal{E}$
$\mathcal{G}^s$=($\mathcal{V}^s, \mathcal{E}^s$)	sparsified conflict graph $\mathcal{G}^s$, composed of a set of vertices $\mathcal{V}^s$ and a set of edges $\mathcal{E}^s$
$h_v(\cdot)$	local threshold function on vertex v
$H(\cdot)$	Heaviside step function
K	The maximum number of control message broadcasting in LGS with fixed scheduling deadline
$\mathcal{L}$	normalized Laplacian matrix of graph $\mathcal{G}$
N	global norm clipping value in Algorithm 2
$\mathbb{N}(a, b)$	normal distribution with mean a & standard deviation b
$\mathcal{N}_{\mathcal{G}}(v)$	set of immediate neighbors of vertex v on graph $\mathcal{G}$
$p(v)$	cut off probability of link v in scheduling
$q(v, t)$, $q(v), \mathbf{q}_v$ $\mathbf{q}$	$q(v, t)$: the queue length of link v at time step t $q(v) = \mathbf{q}_v$: the queue length on link v with t omitted $\mathbf{q}$: the vector of queue lengths on all links $\mathcal{V}$
$r(v, t)$, $r(v), \mathbf{r}_v$, $\mathbf{r}$	$r(v, t)$: link rate of link v at time step t $r(v) = \mathbf{r}_v$: link rate of link v with t omitted $\mathbf{r}$: the vector of link rates for all links $\mathcal{V}$
$u^{(\eta)}$	global threshold for link utility
$u(v), \mathbf{u}_v$ $u(\boldsymbol{v})$	$\mathbf{u}_v = u(v)$: utility value on link v total utility on independent set $\boldsymbol{v}$
$\mathbf{u}$	$\mathbf{u}$: the vector of utility values on all links $\mathcal{V}$
$\mathbb{U}(a, b)$	uniform distribution between a and b
v $\boldsymbol{v}$	$v \in \mathcal{V}$: a vertex (link) on the conflict graph $\mathcal{G}$ (network) $\boldsymbol{v} \subseteq \mathcal{V}$: an independent set on graph $\mathcal{G}$
$\hat{\boldsymbol{v}}$ $\hat{\boldsymbol{v}}^s$ $\tilde{\boldsymbol{v}}^s$	schedule from zero-threshold policy schedule from GCN-based threshold policy schedule from a reference threshold policy
$\mathbf{v}$ $\hat{\mathbf{v}}^s$	$\mathbf{v} \in \{0, 1\}^{\lvert\mathcal{V}\rvert}$: indicator vector of schedule $\boldsymbol{v}$ w.r.t. $\mathcal{V}$ $\hat{\mathbf{v}}^s \in \{0, 1\}^{\lvert\mathcal{V}\rvert}$: indicator vector of schedule $\hat{\boldsymbol{v}}^s$ w.r.t. $\mathcal{V}$
$\mathbf{w}^s$	per-link utility vector under GCN-based threshold policy
$\tilde{\mathbf{w}}^s$	per-link utility vector under a reference threshold policy
$\mathbf{z}, \mathbf{z}(\mathcal{G})$	$\mathbf{z} \in \mathbb{R}^{\lvert\mathcal{V}\rvert}$: node embeddings of $\mathcal{G}$ generated by GCN

In Fig. 3, the performance of the tested threshold policies is presented in relative formats, as approximation ratio (AR) or retention ratio (RR), of which the nominator is a metric of the tested threshold policy and the denominator is that of the zero-threshold policy. In Fig. 3(a), the ARs of the total utility achieved by GCNs and MLP(5) trained by Alt-SGD are well aligned with the statistical baseline (Stat) across all cut-off quantiles η, whereas the GCN(1) trained by ZOO always achieve larger utility at a higher cost for given η, with a

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

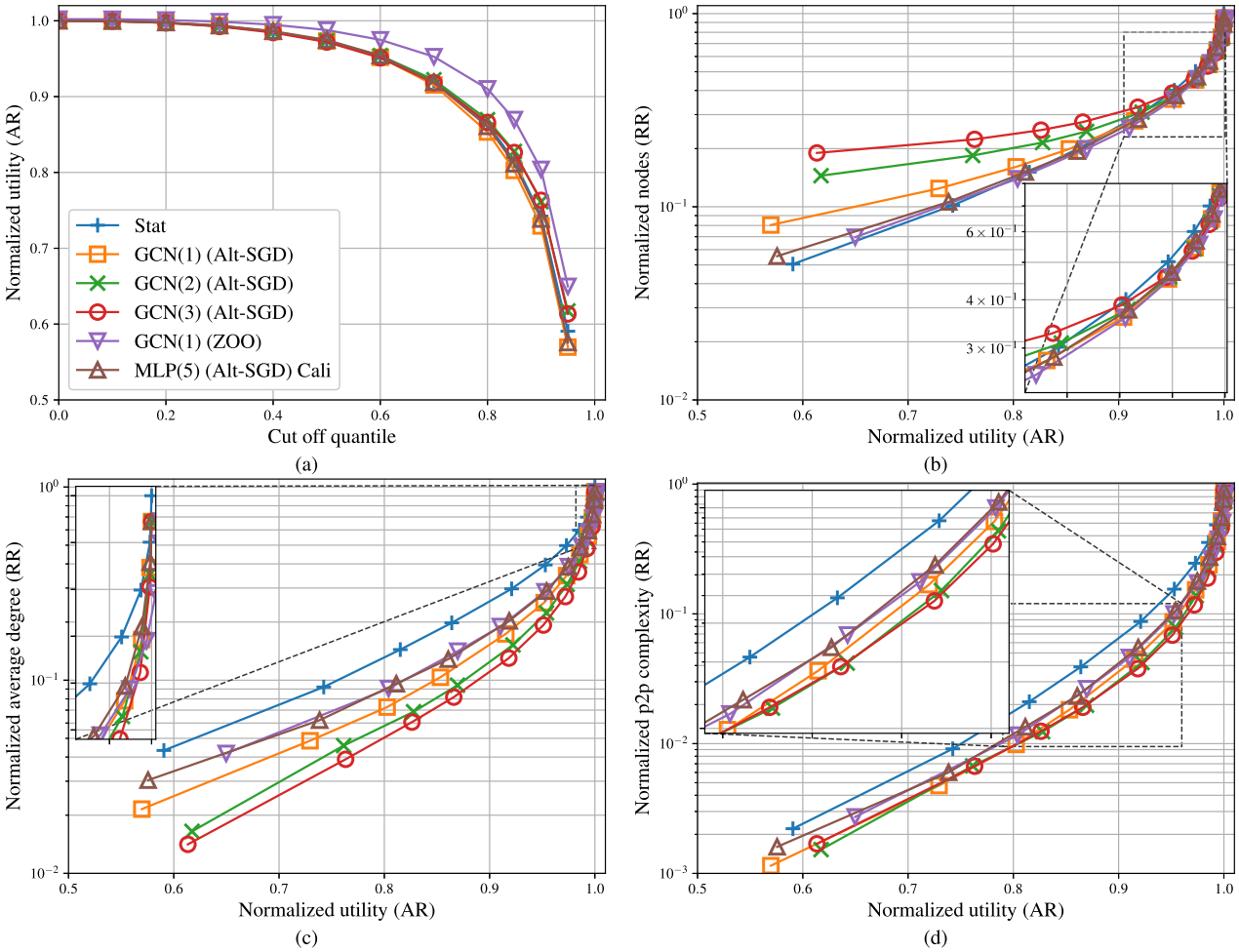


Fig. 3. The performance of different threshold policies normalized by that of zero-threshold policy with identical input network state $(\mathcal{G}, \mathbf{u})$, under vanilla LGS [17]: (a) AR for total utility vs cut-off quantile η showing that Alt-SGD can align GCN-based and the baseline threshold policies on total utilities, (b) RR for number of vertices in $\mathcal{G}^s$ vs normalized utility, (c) RR for average degree of $\mathcal{G}^s$ vs normalized utility, and (d) RR for P2P message complexity of scheduling contention vs normalized utility. For (c) and (d), smaller RR is better.

cost-efficiency similar to MLP(5) as shown next. It demonstrates that Alt-SGD is more effective than ZOO in balancing two competing goals, i.e., minimizing the overhead while achieving required levels of total utility, whereas GCN(1) trained by ZOO would require an additional step of calibration.

To get a deeper insight into the behaviors and cost efficiencies of the tested threshold policies, we plot their RRs of the size of the sparsified conflict graph in Fig. 3(b), the average post-sparsification conflict degree (for $\mathcal{G}^s$), which is linear in the collision rate in CSMA in Fig. 3(c), and their RRs of the point-to-point (P2P) message complexity of LGS in Fig. 3(d), all against the ARs of total utility (normalized utility). The slope of a point in Fig. 3(c) (Fig. 3(d)) represents the overhead cost of achieving an unit of utility under the CSMA (LGS) protocol. Therefore, the lower a curve lies in Figs. 3(c) and 3(d), the more efficient is the corresponding policy. For both CSMA and LGS protocols, all the ML-based threshold policies are more cost-effective than the statistical baseline. Among the ML-based policies, the three GCNs trained by Alt-SGD are more cost-effective than MLP(5) trained by Alt-SGD and GCN(1) trained by ZOO, where the latter two have similar signaling efficiencies. It shows that both the GNN model and Alt-SGD algorithm made meaningful contributions to the performance. Moreover, Alt-SGD is able to exploit the expressiveness of deeper GCNs in learning topological information, thus empowering them to achieve greater efficiencies than the shallower ones, in the region of deep sparsification, e.g., $\eta \geq 0.7$ or normalized utility ≤ 0.95. Fig. 3(b) further shows that under the same normalized utility, deeper GCNs are better at picking links, i.e., retain more links for contention while achieving lower scheduling overhead.

C. Latency in Scheduling Experiments

In link scheduling, identical inputs $\mathbf{u}(t)$ for different sparse schedulers rarely happen because of the dependency between $\mathbf{u}(t)$ and scheduling decisions. To illustrate how the improved signaling efficiency in Section VI-B could be translated to better network performance, we compare six threshold policies with the zero-threshold policy (Dense-Zero) in simulated wireless multi-hop networks under four different distributed scheduling protocols.

The tested threshold policies include 1) Sparse-Stat: global threshold, $u^{(\eta)}$, as the statistical baseline with a given η; 2) Sparse-Stat-(1): global threshold $\mathbb{E}_{v\in\mathcal{V}}(\mathbf{z}_v)u^{(\eta)}$ that takes into account the scaling effect of the 1-layer GCN; 3) Sparse-GCN(L): the GCN-based local threshold policies, with the

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

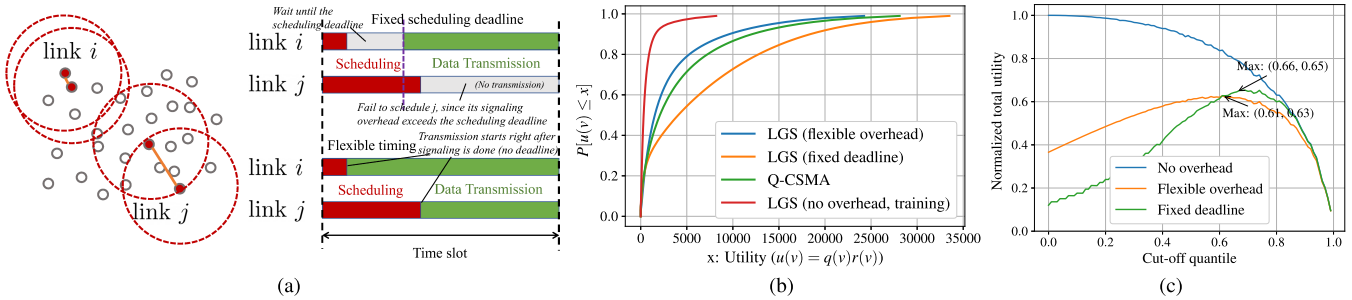


Fig. 4. a) Illustration of two LGS-based protocols: With fixed scheduling deadline, a link will be scheduled only if the signaling meets the deadline, and transmissions start after the deadline; with flexible timing, data transmissions start immediately after the signaling, when all the interfering neighbors of the scheduled links are muted [12]. b) The global distribution (eCDFs) of per-link utility collected from simulation on ER test set under four distributed scheduling protocols: ideal LGS with no overhead, two LGS-based protocols, and Q-CSMA. c) Normalized total utility net of signaling overhead (AR) vs cut-off quantile of global threshold under the two LGS-based scheduling protocols.

GCNs trained by Alt-SGD in Section VI-B; and 4) Hybrid-GCN(1): the hybrid threshold policy described in (7), which enables the GCN(1)-based local threshold on a link v only if its conflict degree $d(v) > D = 25$, so that $D\tau < 30\tau$, the expected scheduling deadline defined later. Hybrid-GCN(1) can demonstrate the performance of our threshold policies in practical LGS-based MaxWeight scheduling.

The first two distributed scheduling protocols are LGS-based MaxWeight schedulers with different ways of splitting a time slot into scheduling and communication sub-slots, as illustrated in Fig. 4(a) [12]. In *LGS with fixed deadline*, scheduling stops by a prescribed deadline, e.g., at 30 ms of a time slot of 100 ms, and only the links that are successfully scheduled by the deadline can use the rest of the time slot, e.g., 70 ms, for transmissions. In *LGS with flexible overhead*, transmission starts right after a link is scheduled and broadcasts a control message to mute its interfering neighbors. Since the interfering neighbors of scheduled links are all muted, such transmission can cause no disruptions. We consider that it takes 1 ms for a link to broadcast a control message to its interfering neighbors, i.e., a full round of message exchange in a neighborhood of 20 links would take 20 ms.

The eCDFs of link utility under zero-threshold policy with ideal LGS scheduler, LGS with fixed deadline and flexible overhead, and Q-CSMA are illustrated in Fig. 4(b), where for the latter three, the realized link rate differs from the nominal one. We train two sets of GCNs based on 1) the ER training set and the utility eCDF for LGS with fixed deadline, and 2) the BA training set and the eCDF for LGS with flexible overhead. The first set of GCNs are then reused in another two scheduling protocols, unweighted CSMA and Q-CSMA in Section VI-C.3, to evaluate their generalizability.

Based on Section IV-D, the optimal cutoff quantiles for the BA test set under LGS with fixed deadline and flexible overhead are estimated as $\eta = 0.66$ and 0.61, respectively, as shown in Fig. 4(c). Notice that there is a deliberate mismatch in interference density $\bar{d}$ between the entire BA test set and our target range $60 \leq \bar{d} \leq 100$ to reflect reality.

1) LGS With Fixed Deadline: In this experiment, the realized link rate is $r(v,t)$ if a link v is scheduled by the deadline (otherwise 0), the traffic setting and BA test set described in Section VI-A are utilized, with a global threshold $u^{(\eta)} = 7912$ for $\eta = 0.66$. The latency performances of tested threshold policies, measured by average backlog across time and networks, as a function of the average conflict degree $\bar{d}$, are shown in Fig. 5(a), and the radio footprints measured by average post-sparsification conflict degree $\mathbb{E}_\Omega[d^s(v)]$ in Fig. 5(b).

With zero-threshold policy (Dense-Zero), latency is low for low conflict densities $\bar{d} \leq 20$, but it increases quickly with $\bar{d}$ due to scheduling failures. In comparison, non-zero threshold policies can retain more transmission opportunities, thus improving the latency when $\bar{d} > 20$. In the target conflict densities ($60 \leq \bar{d} \leq 100$), the global threshold policies Sparse-Stat and Sparse-Stat-(1) have the worst latency performances. The GCN-based threshold policies (Sparse-GCN(L)) can further reduce the backlog over global threshold policies in the target densities, at the cost of slightly worse latency than the global threshold policies for $\bar{d} \leq 20$. Among GCN-based threshold policies, the latency of GCN(2) is slightly better than those of GCN(1) and GCN(3) within the target densities. By leveraging network locality, Sparse-GCN(1) performs better than Sparse-Stat-(1), where the two have the same network-wide average thresholds. Moreover, the hybrid threshold policy (Hybrid-GCN(1)) can achieve low latency across the full range of $\bar{d}$ by taking the best of Dense-Zero and Sparse-GCN(1), demonstrating the practicability of our approach.

Lastly, as shown in Fig. 5(b), the average post-sparsification conflict degrees of GCN-based threshold policies are lower than those of the global threshold policies within the target densities, where GCN(1) achieves the lowest radio footprint for all $\bar{d}$. Overall, GCN(1) can lead to the best latency and radio footprint for networks under LGS with fixed deadline.

2) LGS With Flexible Overhead: In this experiment, the realized link rate is $r(v,t)\,[100 - e(v,t)]\,/70$, where $e(v,t)$ is scheduling overhead of link v at time step t in ms, the traffic setting and BA test set described in Section VI-A are utilized, with a global threshold $u^{(\eta)} = 2310$ for $\eta = 0.61$. The average backlogs under tested threshold policies, as a function of the average conflict degree $\bar{d}$, are presented in Fig. 6(a), and the average post-sparsification conflict degrees in Fig. 6(b).

With the same threshold policy, LGS with flexible overhead achieves lower latency compared to LGS with fixed deadline, as it compresses rather than discards a transmission

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

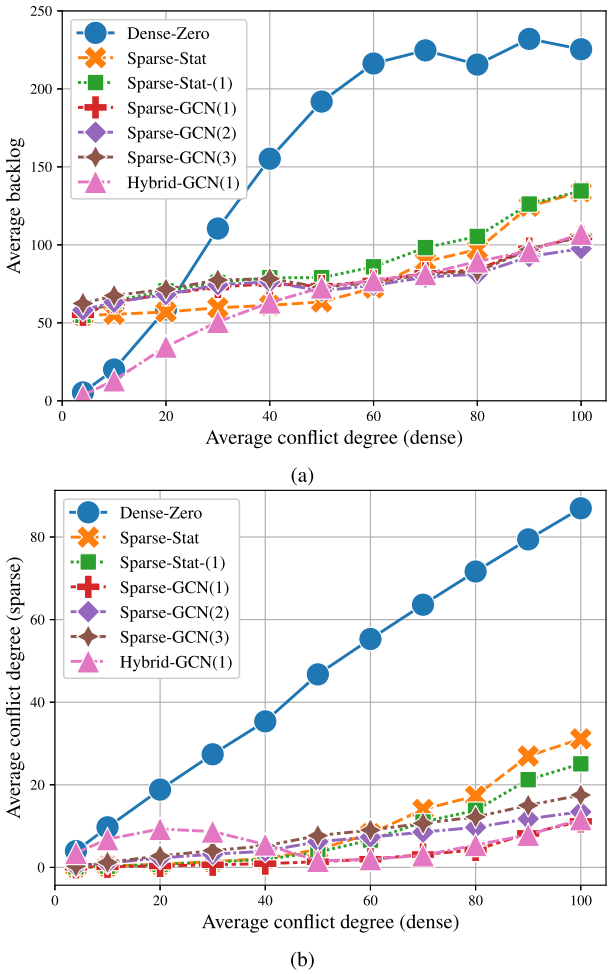


Fig. 5. The performance of threshold policies vs average conflict degree $\bar{d}$ under LGS with fixed deadline: (a) average backlog, and (b) average post-sparsification conflict degree, averaged across time and network. Each point is the average of 60-100 test instances. Global cut-off quantile $\eta = 0.66$. GCNs are trained on ER training set.

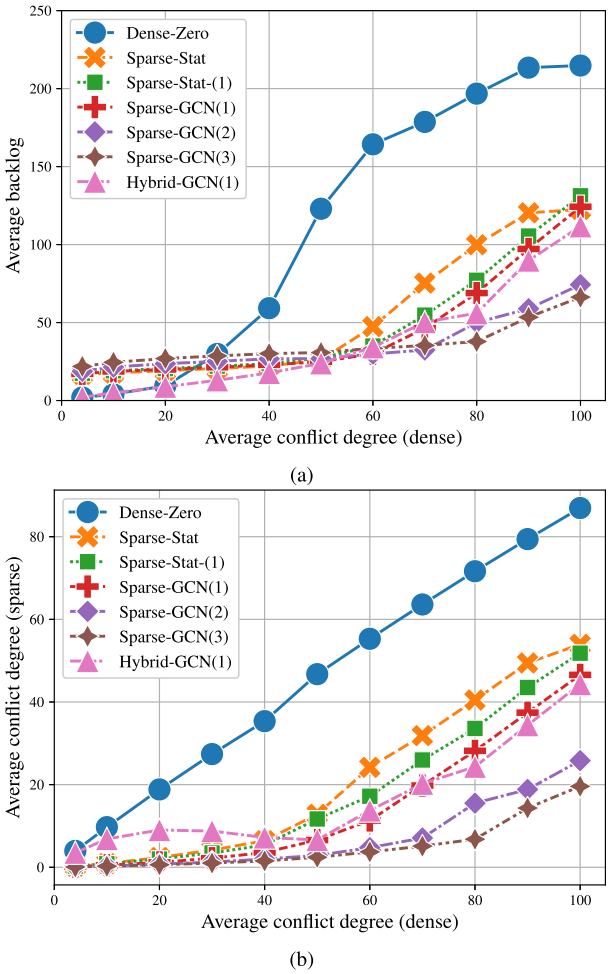


Fig. 6. The performance of threshold policies vs average conflict degree $\bar{d}$ under LGS with flexible overhead: (a) average backlog, and (b) average post-sparsification conflict degree, averaged across time and network. Each point is the average of 60-100 test instances. $\eta = 0.61$. GCNs are trained on BA training set.

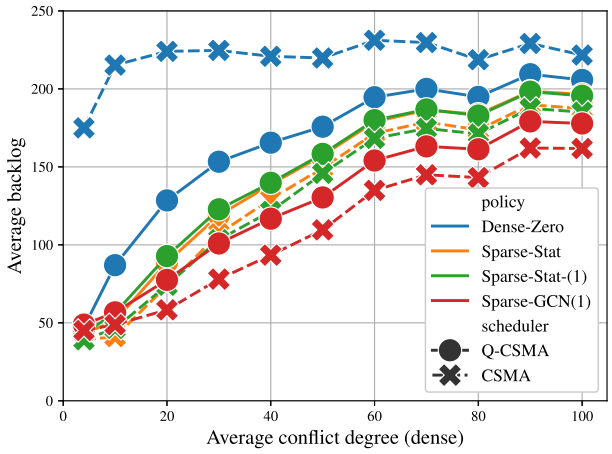
opportunity when the scheduling overhead is high. In the target densities ($60 \leq \bar{d} \leq 100$), threshold policies based on deeper GCNs are ranked the best in both latency and radio footprint, followed by those with shallower GCNs, Sparse-Stat-(1), Sparse-Stat, and zero-threshold policy. The hybrid threshold policy (Hybrid-GCN(1)) outperforms both Sparse-GCN(1) and zero-threshold policy by taking the best of them, except that its radio footprint is higher than that of Sparse-GCN(1) for $\bar{d} < 50$, which demonstrates its practical merit. For networks under LGS with flexible overhead, deeper GCNs always translate to better latency and radio footprint.

3) Q-CSMA and Unweighted CSMA: Unweighted CSMA and Q-CSMA [23] both have a constant signaling overhead and no exponential backoff. In the scheduling sub-slot of Q-CSMA, a link waits a random number of back-off mini-slots in $[0, W-1]$ before sending an INTENT message, and wins the contention if it sends the first INTENT message without collision. The winner of the contention is scheduled with a probability of $e^{u(v)}/(e^{u(v)}+1)$. A link scheduled in time-slot t has priority to transmit in $t+1$, so that it can always deplete its backlog. The unweighted CSMA shares the same procedure except that the winner of the contention is always scheduled. We set the contention window $W = 32$ and a mini-slot to be 1 ms, thus a time-slot of 100 ms contains a scheduling sub-slot of 32 ms and a communication sub-slot of 68 ms.

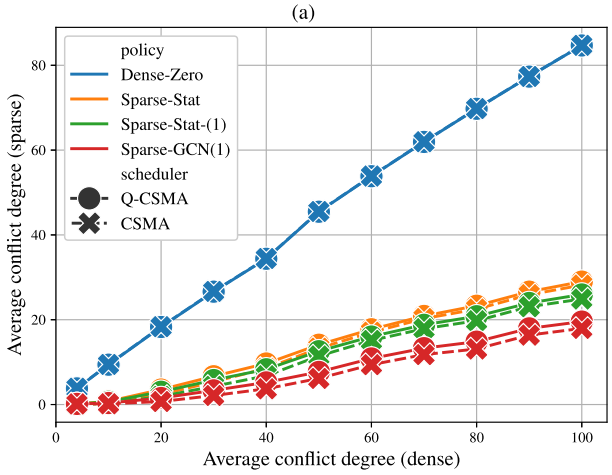
In this experiment, the realized link rate equals the nominal one $r(v,t)$, the traffic setting and BA test set described in Section VI-A are utilized, with a global threshold of $u^{(\eta)} = 5808$ corresponding to $\eta = 0.75$. The average backlogs under four threshold policies with Q-CSMA and unweighted CSMA, as a function of the average conflict degree, are presented in Fig. 7(a), and the average post-sparsification conflict degrees in Fig. 7(b). The GCN(1) model trained for LGS with fixed deadline in Section VI-C.1 is reused for both Q-CSMA and unweighted CSMA. For clarity of presentation, we do not show the results of Sparse-GCN(2) and GCN(3).

In the target densities ($60 \leq \bar{d} \leq 100$), the latency of CSMA-based protocols is worse than those of LGS-based protocols due to increased scheduling failures caused by high collision rate in contention under a constant contention window W. Under Q-CSMA, the largest improvements in latency and radio footprint over zero-threshold policy are brought by Sparse-GCN(1) for around 15%, followed by global threshold policies, Sparse-Stat-(1) and Sparse-Stat, which have similar

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Fig. 7. The performance of threshold policies vs average conflict degree under Q-CSMA [23] and unweighted CSMA protocols specified in Section VI-C.3: (a) average backlog, and (b) average post-sparsification conflict degree, averaged across time and network. Each point is the average of 60-100 test instances. $\eta = 0.75$.

latency but different radio footprints. Although unweighted CSMA has the worst latency given zero-threshold policy, it significantly outperforms Q-CSMA with other non-zero threshold policies in both latency and radio footprint. In particular, with Sparse-GCN(1), the latency of unweighted CSMA is 12% lower than that of Q-CSMA. It suggests that unweighted CSMA with GCN-based threshold policy offers a better alternative to Q-CSMA.

D. Discussions

In Section VI-B, we demonstrated the advantages of GNNs and Alt-SGD over MLP and ZOO in training for signaling efficiency. In Section VI-C, we showed that such efficiency can be translated to improved latency and radio footprint in wireless networks with different protocols. In particular, the benefit of GNNs is two-fold: 1) global scaling on the average threshold in each network to compensate the mismatch between η configured for a set of networks and the true η^* for individual networks, evidenced by the gain of Sparse-Stat-(1) over Sparse-Stat; and 2) exploiting network locality, as shown by the gain of Sparse-GCN(1) over Sparse-Stat-(1). In addition, a trained GCN can generalize to different topologies, and even from LGS to CSMA-based schedulers. Notice that a $3:7$ split of scheduling-communication and a control message overhead as 1% of frame length are chosen to keep simulations tractable on graphs of limited $\bar{d}$. The demonstrated benefits extend to denser networks with other splits and smaller τ.

VII. Conclusion

We propose to reduce the scheduling overhead in dense wireless networks with delay-tolerant traffics, by withholding a link from scheduling contention if its utility is below a threshold. The threshold of each link is individually adjusted by a GCN based on traffic statistics and network locality. Compared to no threshold or a statistical global threshold, GCN-based threshold policy can significantly reduce the overhead of MaxWeight scheduling and collision rate of CSMA in dense wireless networks, leading to improved latency, radio footprint or covertness, and energy efficiency. Moreover, GCNs can be implemented in a fully distributed, asynchronous manner and can generalize well across network topologies and even scheduling protocols. Our 1-layer GCN also has low communication and computing costs thanks to its lightweight neural networks and message passing, and thus can be deployed to low-end wireless devices in MTC and IoT systems. Potential future directions include developing: 1) efficient simulation techniques for large dense networks with higher sparsification ratio and/or random access protocols, 2) online learning approach to replace the offline search of statistical global threshold, and 3) solutions for fairness and tail-delay metrics, as well as for scenarios with higher network mobility and dynamic power-control.

Appendix A
Computation of Partial Derivative in (11)

By substituting $d^s(v)$ from (10) into (9), we get

$$\begin{aligned}
&\mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left(|\mathcal{E}^s||\mathcal{G}\right)\\
&=\frac{1}{2}\sum_{v\in\mathcal{V}}\left[d(v)-d(v)p(v)-\sum_{i\in\mathcal{N}_{\mathcal{G}}(v)}p(i)+\sum_{i\in\mathcal{N}_{\mathcal{G}}(v)}p(v)p(i)\right]\\
&=\frac{1}{2}\left[\sum_{v\in\mathcal{V}}d(v)-2\sum_{v\in\mathcal{V}}d(v)p(v)+\sum_{v\in\mathcal{V}}\sum_{i\in\mathcal{N}_{\mathcal{G}}(v)}p(v)p(i)\right].
\end{aligned}$$

This equation holds because

$$\sum_{v\in\mathcal{V}}\sum_{i\in\mathcal{N}_{\mathcal{G}}(v)}p(i)=\sum_{v\in\mathcal{V}}d(v)p(v),$$

where each node is counted once by each of its neighbors in the left hand side. For each neighbor $i\in\mathcal{N}_{\mathcal{G}}(v)$ of a particular v, $p(v)p(i)$ is counted twice in $\sum_{v\in\mathcal{V}}\sum_{i\in\mathcal{N}_{\mathcal{G}}(v)}p(v)p(i)$. Therefore, the partial derivative of (9) w.r.t. $p(v)$ is

$$\frac{\partial\mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left(|\mathcal{E}^s||\mathcal{G}\right)}{\partial p(v)}=-d(v)+\sum_{i\in\mathcal{N}_{\mathcal{G}}(v)}p(i)=-d^s(v).$$

where we used (10). Based on chain rule, we have

$$\frac{\partial\mathbb{E}_{\Omega_{\mathbf{u}}^{\mathcal{G}}}\left(|\mathcal{E}^s||\mathcal{G}\right)}{\partial z(v)}=-d^s(v)f_u'\left(z(v)u^{(\eta)}\right)u^{(\eta)}.$$

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Peak Search for Quasi-Concave Function

Input: $f_\eta(\cdot), [a,b], 0 < \epsilon \ll 1$
Output: x_m /* *The near optimal,* $|\eta^* - x_m| \leq \epsilon$ */
$x_l = a, x_r = b, x_m = (x_l + x_r)/2$
while $x_r - x_l > \epsilon$ **do**
  $x_1 = \underset{x \in \{x_l, x_r, x_m\}}{\operatorname{argmax}} f_\eta(x)$, $x_2 = \underset{x \in \{x_l, x_r, x_m\} \setminus \{x_1\}}{\operatorname{argmax}} f_\eta(x)$
  $x_3 = (x_1 + x_2)/2$
  if $f_\eta(x_3) \geq f_\eta(x_1)$ **then**
    $x_m \leftarrow x_3$, $x_l \leftarrow \min(x_1, x_2)$, $x_r \leftarrow \max(x_1, x_2)$
  else
    $x_m \leftarrow x_1$, $x_l \leftarrow \min(x_3, x_l)$, $x_r \leftarrow \max(x_3, x_r)$
  end if
end while

APPENDIX B

We solve (6) with Algo. 3, where $f_\eta(\cdot)$ is the objective in (6), $a = 0, b = 1$. It has a complexity of $\mathcal{O}\left(\log\left(\frac{b-a}{\epsilon}\right)\right)$, as the search interval is cut by $1/2$ or $1/4$ in each iteration.

ACKNOWLEDGMENT

The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Office or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for government purposes not with standing any copyright notation herein.

REFERENCES

[1] Z. Zhao, A. Swami, and S. Segarra, "Distributed link sparsification for scalable scheduling using graph neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 5308–5312.
[2] Cisco Systems. (Mar. 2020). *Cisco Annual Internet Report (2018–2023)*. [Online]. Available: https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html
[3] A. Kott, A. Swami, and B. J. West, "The Internet of Battle Things," *Computer*, vol. 49, no. 12, pp. 70–75, Dec. 2016.
[4] I. F. Akyildiz, A. Kak, and S. Nie, "6G and beyond: The future of wireless communications systems," *IEEE Access*, vol. 8, pp. 133995–134030, 2020.
[5] X. Chen, D. W. K. Ng, W. Yu, E. G. Larsson, N. Al-Dhahir, and R. Schober, "Massive access for 5G and beyond," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 3, pp. 615–637, Mar. 2021.
[6] S. K. Sharma and X. Wang, "Toward massive machine type communications in ultra-dense cellular IoT networks: Current issues and machine learning-assisted solutions," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 1, pp. 426–471, 1st Quart., 2020.
[7] X. Lin, N. B. Shroff, and R. Srikant, "A tutorial on cross-layer optimization in wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 24, no. 8, pp. 1452–1463, Aug. 2006.
[8] S. K. Sarkar, T. G. Basavaraju, and C. Puttamadappa, *Ad Hoc Mobile Wireless Networks: Principles, Protocols and Applications*, 2nd ed., Boca Raton, FL, USA: CRC Press, 2013.
[9] N. Patriciello, C. A. Grazia, J. Núñez-Martínez, J. Baranda, J. Mangues-Bafalluy, and M. Casoni, "Performance evaluation of backpressure routing in integrated satellite-terrestrial backhaul for PPDR networks," in *Proc. IEEE 12th Int. Conf. Wireless Mobile Comput., Netw. Commun. (WiMob)*, Oct. 2016, pp. 1–8.
[10] M. Cudak, A. Ghosh, A. Ghosh, and J. Andrews, "Integrated access and backhaul: A key enabler for 5G millimeter-wave deployments," *IEEE Commun. Mag.*, vol. 59, no. 4, pp. 88–94, Apr. 2021.
[11] Z. Zhao, A. Swami, and S. Segarra, "Graph-based deterministic policy gradient for repetitive combinatorial optimization problems," in *Proc. ICLR*, 2023, pp. 1–21.
[12] Z. Zhao, G. Verma, C. Rao, A. Swami, and S. Segarra, "Link scheduling using graph neural networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 3997–4012, Jun. 2023.
[13] C. Joo, X. Lin, and N. B. Shroff, "Understanding the capacity region of the greedy maximal scheduling algorithm in multihop wireless networks," *IEEE/ACM Trans. Netw.*, vol. 17, no. 4, pp. 1132–1145, Aug. 2009.
[14] A. G. Marques, N. Gatsis, and G. B. Giannakis, "Optimal cross-layer design of wireless fading multi-hop networks," in *Cross Layer Designs in WLAN Systems*, N. Zorba, C. Skianis, and C. Verikoukis, Eds., London, U.K.: Troubador Publishing, 2011, pp. 1–44.
[15] S. Basagni, "Finding a maximal weighted independent set in wireless networks," *Telecommun. Syst.*, vol. 18, nos. 1–3, pp. 155–168, Sep. 2001.
[16] W. Cheng, X. Cheng, T. Znati, X. Lu, and Z. Lu, "The complexity of channel scheduling in multi-radio multi-channel wireless networks," in *Proc. IEEE INFOCOM*, Apr. 2009, pp. 1512–1520.
[17] C. Joo and N. B. Shroff, "Local greedy approximation for scheduling in multihop wireless networks," *IEEE Trans. Mobile Comput.*, vol. 11, no. 3, pp. 414–426, Mar. 2012.
[18] C. Joo, X. Lin, J. Ryu, and N. B. Shroff, "Distributed greedy approximation to maximum weighted independent set for scheduling with fading channels," *IEEE/ACM Trans. Netw.*, vol. 24, no. 3, pp. 1476–1488, Jun. 2016.
[19] P. Du and Y. Zhang, "A new distributed approximation algorithm for the maximum weight independent set problem," *Math. Problems Eng.*, vol. 2016, pp. 1–10, 2016.
[20] I. C. Paschalidis, F. Huang, and W. Lai, "A message-passing algorithm for wireless network scheduling," *IEEE/ACM Trans. Netw.*, vol. 23, no. 5, pp. 1528–1541, Oct. 2015.
[21] C. Joo, G. Sharma, N. B. Shroff, and R. R. Mazumdar, "On the complexity of scheduling in wireless networks," *EURASIP J. Wireless Commun. Netw.*, vol. 2010, no. 1, Dec. 2010, Art. no. 418934.
[22] Z. Zhao, G. Verma, C. Rao, A. Swami, and S. Segarra, "Distributed scheduling using graph neural networks," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, Jun. 2021, pp. 4720–4724.
[23] J. Ni, B. Tan, and R. Srikant, "Q-CSMA: Queue-length-based CSMA/CA algorithms for achieving maximum throughput and low delay in wireless networks," *IEEE/ACM Trans. Netw.*, vol. 20, no. 3, pp. 825–836, Jun. 2012.
[24] L. Jiang and J. Walrand, "A distributed CSMA algorithm for throughput and utility maximization in wireless networks," *IEEE/ACM Trans. Netw.*, vol. 18, no. 3, pp. 960–972, Jun. 2010.
[25] X. Lin and S. B. Rasool, "Constant-time distributed scheduling policies for ad hoc wireless networks," *IEEE Trans. Autom. Control*, vol. 54, no. 2, pp. 231–242, Feb. 2009.
[26] M. Luby, "A simple parallel algorithm for the maximal independent set problem," in *Proc. 17th Annu. ACM Symp. Theory Comput.*, 1985, pp. 1–10.
[27] E. Testi and A. Giorgetti, "Blind wireless network topology inference," *IEEE Trans. Commun.*, vol. 69, no. 2, pp. 1109–1120, Feb. 2021.
[28] W. Ye, J. Heidemann, and D. Estrin, "Medium access control with coordinated adaptive sleeping for wireless sensor networks," *IEEE/ACM Trans. Netw.*, vol. 12, no. 3, pp. 493–506, Jun. 2004.
[29] P. Santi, "Topology control in wireless ad hoc and sensor networks," *ACM Comput. Surv.*, vol. 37, no. 2, pp. 164–194, Jun. 2005.
[30] R. Ramanathan, "Antenna beamforming and power control for ad hoc networks," in *Mobile Ad Hoc Networking*, S. Basagni, M. Conti, S. Giordano, and I. Stojmenovic, Eds., Hoboken, NJ, USA: Wiley, 2004, pp. 139–173.
[31] N. K. Ray and A. K. Turuk, "A hybrid energy efficient protocol for mobile ad hoc networks," *J. Comput. Netw. Commun.*, vol. 2016, no. 1, 2016, Art. no. 2861904.
[32] S. Guha, P. B. Basu, C.-K.-C. Chau, and R. Gibbens, "Green wave sleep scheduling: Optimizing latency and throughput in duty cycling wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 29, no. 8, pp. 1595–1604, Sep. 2011.
[33] J. Long and O. Büyüköztürk, "Collaborative duty cycling strategies in energy harvesting sensor networks," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 35, no. 6, pp. 534–548, Jun. 2020.
[34] L. Lin, X. Lin, and N. B. Shroff, "Low-complexity and distributed energy minimization in multihop wireless networks," *IEEE/ACM Trans. Netw.*, vol. 18, no. 2, pp. 501–514, Apr. 2010.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.

[35] X. Xiang, C. Lin, and X. Chen, "Energy-efficient link selection and transmission scheduling in mobile cloud computing," *IEEE Wireless Commun. Lett.*, vol. 3, no. 2, pp. 153–156, Apr. 2014.
[36] W. Wu, W. Xu, Z. Chen, and M. Yang, "Energy-efficient link scheduling in time-variant dual-hop 60GHz wireless networks," *Concurrency Comput., Pract. Exper.*, vol. 32, no. 23, p. 5903, Dec. 2020.
[37] R. Ahlswede, N. Cai, S. Li, and R. W. Yeung, "Network information flow," *IEEE Trans. Inf. Theory*, vol. 46, no. 4, pp. 1204–1216, Apr. 2000.
[38] L. Hai, Q. Gao, J. Wang, H. Zhuang, and P. Wang, "Delay-optimal back-pressure routing algorithm for multihop wireless networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 3, pp. 2617–2630, Mar. 2018.
[39] Z. Wang, J. Cerviño, and A. Ribeiro, "Generalization of graph neural networks is robust to model mismatch," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2025, pp. 2387–2395.
[40] Z. Wang, M. Eisen, and A. Ribeiro, "Learning decentralized wireless resource allocations with graph neural networks," *IEEE Trans. Signal Process.*, vol. 70, pp. 1850–1863, 2022.
[41] S. Liu, P.-Y. Chen, B. Kailkhura, G. Zhang, A. O. Hero III, and P. K. Varshney, "A primer on zeroth-order optimization in signal processing and machine learning: Principals, recent advances, and applications," *IEEE Signal Process. Mag.*, vol. 37, no. 5, pp. 43–54, Sep. 2020.
[42] P.-K. Huang, X. Lin, and C.-C. Wang, "A low-complexity congestion control and scheduling algorithm for multihop wireless networks with order-optimal per-flow delay," *IEEE/ACM Trans. Netw.*, vol. 21, no. 2, pp. 495–508, Apr. 2013.
[43] A. Gupta, X. Lin, and R. Srikant, "Low-complexity distributed scheduling algorithms for wireless networks," *IEEE/ACM Trans. Netw.*, vol. 17, no. 6, pp. 1846–1859, Dec. 2009.
[44] J. Ryu, L. Ying, and S. Shakkottai, "Timescale decoupled routing and rate control in intermittently connected networks," *IEEE/ACM Trans. Netw.*, vol. 20, no. 4, pp. 1138–1151, Aug. 2012.
[45] M. Yadin and P. Naor, "Queueing systems with a removable service station," *J. Oper. Res. Soc.*, vol. 14, no. 4, pp. 393–405, Dec. 1963.
[46] O. Kella, "The threshold policy in theM/G/1 queue with server vacations," *Nav. Res. Logistics*, vol. 36, no. 1, pp. 111–123, Feb. 1989.
[47] D. H. Lee and W. S. Yang, "The n -policy of a discrete time geo/G/1 queue with disasters and its application to wireless sensor networks," *Appl. Math. Model.*, vol. 37, no. 23, pp. 9722–9731, Dec. 2013.
[48] N. Qi, N. I. Miridakis, M. Xiao, T. A. Tsiftsis, R. Yao, and S. Jin, "Traffic-aware two-stage queueing communication networks: Queue analysis and energy saving," *IEEE Trans. Commun.*, vol. 68, no. 8, pp. 4919–4932, Aug. 2020.
[49] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
[50] L. Busoniu, R. Babuska, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Trans. Syst., Man, Cybern., C Appl. Rev.*, vol. 38, no. 2, pp. 156–172, Mar. 2008.
[51] P. Giselsson and A. Rantzer, *Large-Scale and Distributed Optimization*, vol. 2227. Cham, Switzerland: Springer, 2018.
[52] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, 2011, pp. 627–635.
[53] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, 1992.
[54] D. Silver et al., "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016.
[55] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 387–395.
[56] L. Barenboim and M. Elkin, "Distributed graph coloring: Fundamentals and recent developments," *Synth. Lectures Distrib. Comput. Theory*, vol. 4, no. 1, pp. 1–171, 2013.
[57] H. Mousavi, I. S. Amiri, M. A. Mostafavi, and C. Y. Choon, "LTE physical layer: Performance analysis and evaluation," *Appl. Comput. Informat.*, vol. 15, no. 1, pp. 34–44, Jan. 2019.
[58] P. Erdős and A. Rényi, "On random graphs I," *Publ. Math. Debrecen*, vol. 6, nos. 3–4, pp. 290–297, 2022.
[59] A.-L. Barabási and M. Pósfai, *Network Science*. Cambridge, U.K.: Cambridge Univ. Press, 2016.



Zhongyuan Zhao (Member, IEEE) received the B.Sc. and M.S. degrees in electronic engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2006 and 2009, respectively, and the Ph.D. degree in computer engineering from the University of Nebraska–Lincoln, Lincoln, NE, USA, in 2019, under the guidance of Prof. Mehmet C. Vuran. From 2009 to 2013, he worked with ArrayComm and Ericsson as an Engineer, developing 4G base stations. He joined Rice University in 2019 as a Post-Doctoral Research Associate at the Department of Electrical and Computer Engineering advised by Prof. Santiago Segarra. Currently, he is a Research Assistant Professor at Rice University. His current research interests include machine learning, signal processing, and dynamic resource allocation in wireless communications and networked systems.



Gunjan Verma (Member, IEEE) received the B.S. degree in mathematics, computer science, and economics from Rutgers University, the M.A. degree in computational biology from Duke University, and the M.S. degree in mathematics from Johns Hopkins University. He is currently a Senior Computer Scientist and the Research and Development Portfolio Director of the U.S. Army Research Laboratory (ARL), Adelphi, MD, USA. His research interests include Bayesian statistics and machine learning and their application to problems in networking, communications, and robotics. In 2025, he received the Civilian Service Commendation Medal from the U.S. Department of the Army in recognition of his exceptional leadership, research, and development efforts in several high-impact research and development initiatives.



Ananthram Swami (Life Fellow, IEEE) received the B.Tech. degree from IIT Bombay, the M.S. degree from Rice University, and the Ph.D. degree from the University of Southern California (USC), all in electrical engineering. He is currently with the U.S. Army's DEVCOM Army Research Laboratory (ARL) as the Army's Senior Research Scientist (ST) for network science and is an ARL Fellow. Prior to joining ARL, he held positions with Unocal Corporation, USC, and CS-3. He has held visiting faculty positions at INP, Toulouse, and Imperial College, London. His recent awards include the 2024 IEEE LANMAN and 2024 IEEE ICC Best Paper Awards, the 2018 IEEE ComSoc MILCOM Technical Achievement Award, and the 2017 Presidential Rank Award (Meritorious). He is a Co-Editor of the 2023 book *IoT for Defense and National Security* (IEEE/Wiley) and co-author of the book *Network Tomography* (Cambridge University Press, 2021).



Santiago Segarra (Senior Member, IEEE) received the B.Sc. degree (Hons.) (Valedictorian) in industrial engineering from the Instituto Tecnológico de Buenos Aires (ITBA), Argentina, in 2011, and the M.Sc. degree in electrical engineering and the Ph.D. degree in electrical and systems engineering from the University of Pennsylvania (Penn), Philadelphia, PA, USA, in 2014 and 2016, respectively. From September 2016 to June 2018, he was a Post-Doctoral Research Associate with the Institute for Data, Systems, and Society at Massachusetts Institute of Technology. In 2018, he joined the Department of Electrical and Computer Engineering, Rice University, as an Assistant Professor, and is currently an Associate Professor. He also holds courtesy appointments with the Departments of Computer Science and Statistics. His research interests include network theory, data analysis, machine learning, and graph signal processing. He received the 2011 Outstanding Graduate Award granted by the National Academy of Engineering of Argentina, the 2017 Penn's Joseph and Rosaline Wolf Award for Best Doctoral Dissertation in Electrical and Systems Engineering, the 2020 IEEE Signal Processing Society Young Author Best Paper Award, the 2021 Rice's School of Engineering Research + Teaching Excellence Award, three early career awards (NSF CAREER, ARO ECP, and ARI Early Career), and five best conference paper awards. Since 2022, he has dedicated part of his time consulting for Microsoft Research.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 26,2026 at 11:35:33 UTC from IEEE Xplore. Restrictions apply.